

Digital Forensics Basics: A Step-by-Step Guide for Beginners

An Open-Source Approach

Weifeng Xu, Ph.D.; Lin Deng, Ph.D.; and Mohit
Ajaykumar Dhabuwala

© 2026 *Dr. Xu's Research Lab*, Baltimore, Maryland
ALL RIGHTS RESERVED.

2026, Second Edition
E-book created September 14, 2026

Dedication

To my incredible research team, whose unwavering dedication, expertise, and collaborative spirit made this book possible. Your hard work, insights, and commitment to excellence have been the foundation of this project. Thank you for your patience, creativity, and tireless efforts in pushing the boundaries of knowledge. This book is as much yours as it is mine.

Introduction

“Míng chá qiū háo, jiàn wēi zhī zhù.”

“To perceive the finest detail of an autumn hair is to understand the greater truth it reveals.” Just as ancient sages sought wisdom in the smallest of signs, the art of forensic investigation lies in uncovering the subtle truths hidden within the minutiae of data. Every fragment, no matter how small, holds the key to unraveling the larger story.

—Ancient Chinese Proverb

Preface

Digital evidence now shapes criminal investigations, incident response, corporate inquiries, and everyday decisions about security and privacy. Yet beginning students often encounter digital forensics as a collection of unfamiliar tools, commands, and screenshots. This book was written to provide a more coherent starting point: an accessible, open-source, hands-on introduction that connects each technical action to an investigative question and to the evidence needed to answer it.

The book begins with one foundation chapter and continues with thirteen instructional chapters. The foundation leads directly to a first investigation of a supplied USB image in Autopsy. Readers then study number systems, computer-system context, and Linux skills before progressing to file-system analysis, evidence acquisition, evidence searching, data carving, network traffic analysis, and concealed data in images. The goal is not to make readers memorize a single tool or procedure. It is to help them develop habits that remain useful as tools change: preserve original evidence, work from verified copies, record what was done, interpret output cautiously, and distinguish an observation from the conclusion it may support.

The chapters are organized around connected laboratory scenarios and evidence questions rather than isolated software demonstrations. A USB image, a deleted file, a network capture, or a suspicious image becomes meaningful only when it is examined in context and linked to a defensible explanation. This organization lets readers see how foundational skills support later work: acquisition enables analysis, analysis creates leads, and multiple artifacts can corroborate or challenge an interpretation.

This text is intended for undergraduate and graduate students, instructors, self-directed learners, and early-career practitioners who want a practical entry point into digital forensics. It is suitable for introductory digital-forensics courses, cybersecurity and criminal-justice technology programs, and professional training

that needs guided laboratory activities. Readers do not need deep prior forensic experience, but they should be prepared to work carefully, ask questions about evidence, and practice in authorized environments.

The official book companion site at <https://github.com/frankwxu/Digital-Forensics-Basic-Book> provides chapter-aligned presentations, updates, and related supporting information. We encourage instructors to adapt the labs to their course objectives and learners to repeat them with small, documented variations. When an activity requires a training file, the relevant chapter provides the instructions and link needed to obtain it.

For practical guidance on navigating chapters, reading commands and screenshots, using the companion materials, and studying effectively, begin with Chapter , *How to Use This Book*. Chapter 1 then establishes the disciplinary foundation of digital forensics and its relationship to cybersecurity. We hope this sequence helps readers approach each lab not as a script to copy, but as an opportunity to practice careful, reproducible forensic reasoning.

Weifeng Xu, Ph.D.
Lin Deng, Ph.D.
Mohit Ajaykumar Dhabuwala
Baltimore, Maryland
2026

Contents

Dedication	ii
Introduction	iii
Preface	iv
How to Use This Book	xvi
1 Who This Book Is For	xvi
2 How the Chapters Work	xvi
3 Tools and Laboratory Environments	xvii
4 Your Learning Path Through the Book	xviii
5 Companion Materials	xix
6 Reading the Commands, Screenshots, and Notes	xix
7 How to Get the Most from the Labs	xx
1 Introduction to Digital Forensics	1
1.1 Introduction to digital forensics	2
1.1.1 Why digital forensics matters	2
1.1.2 Core goals of a digital forensic investigation	3
1.1.3 Digital Forensics versus Data Recovery	3
1.1.4 Types of digital evidence	4
1.2 Digital Investigation Workflow	4
1.2.1 Gathering Evidence and Initial Custody	5
1.2.2 Acquisition: Forensic Images versus Backup Copies	8
1.2.3 Examination and Analysis	9
1.2.4 Reporting and Case Critique	9
1.3 Digital Forensics and Related Disciplines	10
1.3.1 Public- and Private-Sector Digital Investigations	11
1.4 Summary	12

2	Your First Investigation: USB Image Analysis with Autopsy	15
2.1	Investigating a USB Disk Image with Autopsy	16
2.1.1	Introduction to Autopsy	16
2.1.2	Preparing the Autopsy environment and evidence image	16
2.2	Analyzing a USB image with Autopsy	20
2.2.1	Creating a New Case	20
2.2.2	Adding the USB image as a data source	23
2.3	Recovering the deleted content from the USB image	26
2.3.1	Finding the files deleted from the USB	26
2.3.2	Tagging deleted files for review	26
2.3.3	Recovering and extracting the tagged files	30
2.4	Generating the final forensic report	31
2.5	Summary	34
3	Number Systems for Digital Forensics	36
3.1	Number system conversions in digital forensics	37
3.1.1	Linux-Based Number System Conversions	37
3.1.2	Python-Based Number System Conversions	42
3.2	ASCII: Bridging human-readable text and binary code	43
3.2.1	Understanding Character Encoding: ASCII vs. UTF-8	44
3.2.2	Linux ASCII Conversions	46
3.3	ASCII Conversions in Python 3	47
3.4	Understanding UNIX time: A numerical representation of time	48
3.5	UNIX Time (Python version)	50
3.5.1	Getting Current Time and Timestamps	50
3.5.2	Converting Timestamps to Readable Time	52
3.5.3	Comparing Epochs and Time Differences	54
3.6	Understanding hash codes in investigations	55
3.6.1	Command-Line Hashing	55
3.6.2	Python Hashing	56
3.7	The significance of byte order in digital forensics	57
3.7.1	Byte order identification	57
3.7.2	Contrasting byte ordering schemes: Little Endian vs. Big Endian	58
3.8	Summary	59

4	Computer Systems and Digital Evidence	61
4.1	Computer systems as evidence environments	61
4.2	Core components and their forensic relevance	63
4.3	Runtime system context and evidence correlation	66
4.4	Hard disks as forensic evidence sources	69
4.5	Partitions, volumes, and drive letters	72
4.6	Booting process and partition-table overview	76
4.7	Master Boot Record	78
4.8	GUID Partition Table	85
4.9	Summary	91
5	Examining Digital Evidence with Linux	93
5.1	Introduction to Operating System (OS)	94
5.2	Introduction to File System	96
5.3	The Virtual File System: A universal translator for file systems .	98
5.4	Navigating the Linux command line	101
5.4.1	Getting started in the terminal	101
5.4.2	Exploring files and directories from the terminal	106
5.5	Understanding inodes in Linux	111
5.5.1	Inspecting inode and filesystem metadata	111
5.6	Operations on files and directories	116
5.6.1	Creating and inspecting files and directories	116
5.6.2	Copying, moving, and deleting files	119
5.7	Understanding and managing the PATH variable	123
5.7.1	Working with file paths and command lookup	124
5.8	Finding specific text within files	128
5.8.1	Using grep to search text and files	128
5.9	Investigating network activity in Linux	132
5.9.1	Testing connectivity and inspecting ports	132
5.9.2	Downloading and examining retrieved files	137
5.10	Creating shell scripts	140
5.10.1	Writing and inspecting a simple shell script	141
5.11	Modifying file permissions with the chmod command	143
5.12	Summary	146

6	Automating and Scaling Forensic Examination with Linux	149
6.1	Input and Output Devices as Files	149
6.1.1	Understanding the default input stream: stdin	151
6.1.2	Understanding the default output stream: stdout	152
6.1.3	Handling and redirecting the error messages	155
6.2	Communication and Data Transfer in Linux for Digital Forensics	156
6.3	Making cross-platform data transfers	160
6.3.1	Transferring files from Kali Linux to Windows 10	160
6.3.2	Create a backdoor on Windows 10 / remote file acquisition	164
6.4	Reverse Shells: A Forensic Perspective	167
6.5	Bypassing firewalls with outbound connections	168
6.6	Remote disk acquisition in digital forensics	168
6.6.1	Key considerations for remote forensic imaging	169
6.6.2	Assessing Disk Space Availability for Acquisition	170
6.6.3	Remote Disk Acquisition: Linux - receiving the image . .	172
6.6.4	Acquiring Windows disk partition remotely	173
6.6.5	Verifying the file transfer	176
6.7	Addressing slow remote disk acquisition	177
6.7.1	Automating the Experimentation (Linux Only)	179
6.7.2	Manual Block Size Testing	181
6.8	Summary	182
7	File-System Evidence Analysis with The Sleuth Kit	184
7.1	Analyzing the forensic image: Autopsy and The Sleuth Kit . . .	185
7.2	File system and its layers	189
7.3	The Host Protected Area (HPA)	191
7.4	Examining a USB drive using TSK	193
7.4.1	Preparing the USB image for analysis	193
7.4.2	Identifying the image and file system	196
7.4.3	Listing and recovering deleted files	198
7.4.4	Examining raw blocks and hexadecimal content	201
7.5	Accessing USB Image Content	204
7.6	Summary	207
8	Acquiring and Verifying USB Evidence	209
8.1	Understanding disk images	210
8.1.1	File formats for disk images	210

8.1.2	Image acquisition	210
8.2	USB image acquisition in Windows	211
8.2.1	Acquiring a USB image with FTK Imager	211
8.3	Setting up a Kali Linux virtual machine on a Windows host . . .	217
8.3.1	Preparing the Kali virtual machine	218
8.4	Imaging a USB drive in Linux	221
8.5	Image acquisition over the network	223
8.5.1	Kali as sender, Windows as receiver	223
8.5.2	Windows as sender, Kali as receiver	226
8.6	Summary	228

9 Digital Evidence Search: The Power of Regular Expressions in Digital Forensics 231

9.1	What is Regular Expression (regex)?	232
9.1.1	Understanding the Grep command and its role in pattern matching	232
9.1.2	The regex cheatsheet	233
9.2	Hands-on regex lab	233
9.2.1	Preparing the lab files	235
9.2.2	First searches in the roster files	237
9.3	Word character, word string, and word boundary	240
9.3.1	Two-character words and word boundaries	241
9.3.2	Whole-word matching on realistic data	244
9.4	Conditional matching in regex: Lookaround assertions	248
9.4.1	Lookbehind assertions	249
9.4.2	Lookahead assertions	250
9.4.3	Mismatch in lookups	252
9.4.4	Getting exact matches	252
9.5	Matching phone numbers	254
9.5.1	Matching any 10-digit number	254
9.5.2	Matching exact 10-digit number	255
9.5.3	Matching phone numbers with period separators	256
9.5.4	Matching phone numbers with or without period separators	257
9.5.5	Matching phone numbers with multiple separators	257
9.5.6	Matching Phone Numbers with Multiple Formats	259
9.6	Matching email addresses and passwords	260
9.6.1	Matching a basic email pattern	261

9.6.2	Matching email addresses in the text file	262
9.6.3	Check if a password is valid	263
9.7	Applying Regex on Java source code	264
9.7.1	Search string in Java and show line numbers	264
9.7.2	Counting keyword occurrences in Java source code	265
9.8	Extracting artifacts from HTML, XML, and document formats	267
9.8.1	Extracting HTML tags and content with backreferences	268
9.8.2	Matching HTML tags using Lazy quantifier	269
9.8.3	Extracting Content within HTML Tags	270
9.9	Matching network and low-level artifacts	272
9.9.1	Matching IPv4 addresses with groups	272
9.9.2	Matching HTTP requests and URLs	273
9.9.3	Extracting hexadecimal numbers	276
9.9.4	Matching MAC addresses in text	277
9.9.5	Matching MAC addresses from a PCAP file	279
9.9.6	Analyzing XML inside .docx files	283
9.9.7	Using sed for document cleanup and extraction	287
9.10	Searching disk images with find and metadata	298
9.10.1	Understanding the find command for evidence search	299
9.10.2	Preparing and examining the evidence image	299
9.10.3	Advanced file search techniques with the find command	304
9.10.4	Pattern-based filename searches with find	312
9.10.5	Filtering file paths with find's -regex option	315
9.11	Filtering files by permissions, timestamps, size, and ownership	320
9.11.1	Identifying files with full permissions	321
9.11.2	Identifying files without full permissions	322
9.11.3	Locating files with specific permissions	323
9.11.4	Understanding file timestamps: access time, modify time, and change time	325
9.11.5	Using find to identify files based on timestamps	334
9.11.6	Finding files accessed within a specific timeframe	337
9.11.7	Using find to identify files based on size	338
9.11.8	Locating files belonging to a specific user	343
9.12	Anti-Forensics: The challenge of timestamp manipulation	344
9.12.1	Setting fake timestamps by changing system time	345
9.12.2	Restoring system time to the correct value	346
9.12.3	Setting a fake time using touch -t	347

9.12.4	Copying files and preserving timestamps	350
9.12.5	Examining command history for forensic clues	351
9.13	Summary	353

10 Data Carving: Recovering Lost and Hidden Data 355

10.1	Accessing data units (Blocks) of a file in Linux EXT4 file system	356
10.1.1	Downloading the .JPG image file	356
10.1.2	Examining file metadata and block allocation	358
10.2	Getting file information and signature of a .jpg file	359
10.2.1	Examining the internal structure of JPEG file	361
10.2.2	Image compression technique: Huffman Coding	362
10.3	Exploring the xxd Utility for Forensic Analysis	364
10.3.1	Understanding the file content structure	365
10.3.2	Examining JPEG File Signatures with xxd	366
10.3.3	Alternative method of viewing JPG header and tail signatures	367
10.3.4	Viewing specific portions of a file	368
10.4	Utilizing hexadecimal analysis for file type identification	369
10.4.1	Generating plain hex dumps	369
10.4.2	Searching for signatures in plain hex dumps	370
10.5	Introduction to the tr (translate) command	371
10.5.1	Implementing tr command for text manipulation	371
10.5.2	Removing newlines (\n) from hex dumps	372
10.5.3	Restructuring text with tr	373
10.5.4	Matching content between header and tail signatures . . .	377
10.5.5	Reversing a hex dump to its original binary form	378
10.6	Interesting evidence in file header	379
10.6.1	Finding the author information	379
10.6.2	Viewing some specific metadata	381
10.6.3	Viewing author name and offset	382
10.7	Fake the date (2020 - 2030)	382
10.8	Extracting images from a word document	384
10.8.1	Preparing and downloading the lab files	385
10.8.2	Installing Bless Hex Editor with Flatpak	386
10.9	Analyzing the hex dump in bless hex editor	389
10.9.1	Opening the document in Bless	389
10.9.2	Locating the embedded JPEG boundaries	390
10.9.3	Extracting, saving, and viewing the carved image	392

10.10	Using alternative approaches for file carving	397
10.10.1	File carving with the xxd command	397
10.10.2	File carving with the binwalk command	399
10.10.3	File carving with the binwalk -e command	401
10.11	Carving/Recovering a USB image	402
10.11.1	Prepare a USB image for file carving	402
10.11.2	Viewing the content of the zipped file	404
10.11.3	Extracting the contents of the 7z archive	405
10.11.4	Verifying the integrity of the extracted USB image file . .	406
10.12	Examining the extracted USB content	407
10.12.1	Identifying partition structure	408
10.12.2	Selecting file types for carving	409
10.12.3	Identifying and listing deleted files on the USB image . .	410
10.13	Introduction to scalpel command	413
10.13.1	Accessing the Scalpel help page	413
10.13.2	Carving files from the USB image	414
10.13.3	Viewing the carved output as a directory tree	416
10.13.4	Reviewing the Scalpel audit log	417
10.13.5	Interpreting the carved images	419
10.14	Summary	421

11 Wireshark for Digital Investigations - Part 1: Capturing, Filtering, and Examining HTTP and TCP Communications 423

11.1	Understanding Wireshark's command-line options	424
11.2	Capturing and analyzing web traffic with Wireshark	425
11.2.1	Creating a simple HTML web page	425
11.2.2	Starting Apache and testing the HTML page	426
11.2.3	Modifying the HTML file permission	427
11.2.4	Retrieving HTML file information with the curl command	428
11.3	Capture HTTP traffic using Wireshark	430
11.3.1	Preparing Wireshark for non-root capture	430
11.3.2	Preparing the browser and capture files	434
11.3.3	Opening a saved capture or starting live capture	435
11.4	Establishing TCP connections: The three-Way handshake	438
11.4.1	Capturing the SYN packet: Initiating the TCP handshake	439
11.4.2	Capturing the SYN-ACK packet: Server's acknowledgment and response	440

11.4.3	Capturing the ACK packet: Finalizing the TCP handshake	442
11.5	Analyzing HTTP requests	443
11.5.1	Filtering and analyzing HTTP Requests	444
11.5.2	TCP segment and payload analysis of an HTTP request	445
11.5.3	Analyzing IP and MAC address details within the HTTP request	447
11.5.4	Acknowledgment (ACK) packet in the HTTP request	449
11.6	Examining the HTTP response	451
11.6.1	Dissecting the captured response	451
11.6.2	Examining the HTTP response data	452
11.6.3	Analyzing the TCP segment of the HTTP response data	453
11.6.4	Analyzing the client's TCP acknowledgment	455
11.7	Ending the TCP connection	456
11.7.1	Client-initiated TCP connection termination	457
11.7.2	Server acknowledgment and connection closure	458
11.8	Summary	459

12 Wireshark for Digital Investigation - Part 2: Image Extraction 461

12.1	Capturing and examining HTTP traffic with embedded image	462
12.1.1	Using the provided capture	462
12.1.2	Acquiring the Wireshark capture log file	462
12.2	Analyzing the HTTP GET request and response	463
12.2.1	Analyzing the HTTP GET image request and response	464
12.2.2	Calculating the length of the TCP segments	465
12.2.3	Dissecting the raw data of TCP Payload	467
12.2.4	Determining the HTTP response header length	468
12.2.5	Identifying image boundaries within TCP segments	469
12.3	Extracting image from the HTTP GET request	471
12.3.1	Exporting the packet byte containing the .JPG file	471
12.3.2	Saving the exported packet as .jpg file	473
12.3.3	Visualizing the extracted image	474
12.3.4	Documenting the recovered image	475
12.4	Relationship between IP packet and TCP payload	475
12.4.1	Dissecting the structure of TCP (Transmission Control Protocol) header	476
12.4.2	Understanding the IP (Internet Protocol) header structure	477

12.4.3	Examining the relationship between IP Packet and TCP payload size	479
12.5	Summary	481
13	Detecting and Analyzing Concealed Data in Digital Images	483
13.1	Steganography Techniques: Substitution	484
13.2	Hands-on with steganography techniques	488
13.2.1	Acquiring the .bmp Image	488
13.2.2	Viewing the downloaded image	489
13.2.3	Viewing the Steghide help page	490
13.2.4	Creating the secret message text file	491
13.3	Embedding data within an image using Steghide	492
13.3.1	Embedding a text file within an image	493
13.3.2	Extracting embedded data with Steghide	494
13.4	Cracking the password using a wordlist	495
13.5	Summary	497
14	Conclusion and Next Steps in Digital Forensics	499
14.1	Bringing the Skills Together	500
14.2	What a Careful Examiner Must Remember	500
14.3	Ethics, Accuracy, and Professional Responsibility	501
14.4	Using the Companion Materials to Keep Learning	502
14.5	Where to Go Next	502
14.6	Summary	503
	The Author: Weifeng Xu, Ph.D.	505
	The Author: Lin Deng, Ph.D.	506
	Glossary	507
	Bibliography	514

How to Use This Book

This opening guide explains how to navigate the book and work effectively with its labs. The chapters that follow are practical, tool-driven, and evidence-focused, using command-line workflows, screenshots, sample artifacts, and short investigative exercises. Read this guide first, then begin Chapter 1 for the conceptual foundation of digital forensics and its relationship to cybersecurity.

1 Who This Book Is For

This book is intended for students, instructors, and self-directed learners who want a practical introduction to digital forensics. It is especially useful for readers who learn best by doing: running commands, examining files, comparing outputs, and connecting technical observations to investigative meaning.

The book assumes curiosity and persistence more than deep prior expertise. After the introduction, Chapter 2 offers a first investigation using Windows and Autopsy with a supplied USB image. It introduces the terms needed for the lab without requiring Linux experience. Number systems, computer systems, and Linux follow, explaining the foundations for deeper analysis, network traffic review, data carving, and steganography. If a chapter feels advanced at first, use the earlier chapters and the companion materials as a bridge rather than treating each topic as isolated.

2 How the Chapters Work

Most chapters follow a recurring pattern:

Learning Objectives Each chapter opens with measurable goals that identify the concepts, evidence types, and practical skills you should be able to explain

or apply.

Orientation A short conceptual opening explains what the tool, artifact, or forensic method is and why it matters.

Hands-on workflow The chapter then shifts into practical steps using commands, screenshots, and lab files.

Interpretation Important outputs are connected back to a forensic question, such as what was accessed, when an action occurred, what data was hidden, or how a file can be recovered.

Summary Each chapter closes by reinforcing the main investigative lessons and the role of the technique in real casework.

Chapter Review Review questions, discussion prompts, and a hands-on exercise help you check understanding, reason through investigative tradeoffs, and apply the chapter's methods.

Readers will get the most value from the book by treating each chapter as an investigation rather than a script to copy mechanically. When you see a command, pause long enough to ask what evidence it is meant to reveal, what part of the output matters, and how that output supports a forensic conclusion.

3 Tools and Laboratory Environments

Forensic tools help acquire data, calculate hashes, parse artifacts, recover deleted files, inspect network traffic, and organize case information. Tools do not replace judgment. Their output must be validated, understood, and interpreted in the context of the case.

Use a controlled laboratory environment for practice. Work with copies of training images rather than original media, record the tool version and settings, and retain enough notes for another reader to reproduce the result. The chapters in this book introduce Linux utilities, Autopsy, The Sleuth Kit, Wireshark, regular expressions, data carving, and image-steganography techniques through that practical lens.

4 Your Learning Path Through the Book

The chapters are designed as connected stages of an investigation rather than independent tool tutorials. The following roadmap shows the question each chapter helps an investigator answer.

Chapter	Connection to the forensic workflow
Chapter 2	Examine a USB image through Autopsy and connect artifacts to investigative questions.
Chapter 3	Interpret raw values, timestamps, encodings, hashes, and byte order accurately.
Chapter 4	Identify how computer-system layers, storage layout, and device context shape the available evidence.
Chapter 5	Navigate a forensic workstation and inspect evidence with foundational Linux tools.
Chapter 6	Use more advanced Linux commands to make examination and analysis more efficient.
Chapter 7	Perform transparent command-line file-system analysis with The Sleuth Kit.
Chapter 8	Preserve removable-media evidence by creating and verifying a forensic image.
Chapter 9	Search large evidence collections precisely with regular expressions.
Chapter 10	Recover content when file-system metadata is missing, deleted, or damaged.
Chapter 11	Analyze network evidence by capturing, filtering, and examining communications.
Chapter 12	Extract image evidence from captured network traffic.
Chapter 13	Identify and investigate data concealed inside digital images.
Chapter 14	Consolidate the techniques and plan the next steps in forensic practice.

The book moves from principles to a first investigation: Chapter 2 applies Chapter 1’s evidence practices to a supplied USB image in Autopsy. Chapters 3 and 4 then explain the representation and computer-system concepts encountered in that lab. The Linux chapters build command-line skills for deeper examination,

followed by file-system analysis, acquisition, and investigations of files, network traffic, and concealed data.

As you work through the book, keep three questions in mind: What does this artifact show? How reliable is that interpretation? What additional evidence would confirm or challenge it? Those questions turn tool output into defensible forensic reasoning.

5 Companion Materials

The official book companion site is <https://github.com/frankwxu/Digital-Forensics-Basic-Book>. It provides chapter-aligned presentations, updates, and related supporting information.

The shared, scenario-based approach used by these materials is described in Xu et al. [2022].

When a chapter asks you to download a file, inspect a sample artifact, or compare your results to a screenshot, follow the instructions and link provided in that chapter. Use the companion site to locate the matching presentation and other reader resources. Using these materials with the book will make the learning process smoother and will help you reproduce the examples more reliably.

6 Reading the Commands, Screenshots, and Notes

This book uses a consistent hands-on style. To follow it more easily, keep these conventions in mind:

Commands Commands shown in bold or monospaced form are meant to be typed exactly unless the text tells you to adapt a filename, path, or interface name.

Screenshots Screenshots are not decorative. They show what you should expect to see after a step succeeds and often highlight the part of the output that matters most.

Artifacts Filenames, hashes, packet captures, image files, and directories are

evidence-bearing items. Pay attention to their names and locations because later commands often build on earlier artifacts.

Interpretation A command by itself is not the goal. The goal is to understand what the result means in an investigative context.

Warnings and notes Short notes are included to help you avoid common mistakes, recognize environment differences, and distinguish normal behavior from suspicious evidence.

Many chapters assume a Linux-based lab workflow, often with Kali Linux or another forensic workstation environment. If your local paths, permissions, browser cache locations, or installed tools differ from the examples, focus first on the investigative purpose of the step and then adapt the command as needed for your environment.

7 How to Get the Most from the Labs

The most effective way to use this book is to work actively. Type the commands, inspect the outputs, and compare what you see to the figures. If a result differs from the screenshot, do not assume failure immediately; instead, ask whether the difference comes from timestamps, permissions, software versions, network state, or changes in the sample data.

It is also helpful to keep brief notes while you work. Record the commands you ran, what evidence they exposed, and what conclusion each step supported. That habit will make the later reporting chapters more meaningful and will strengthen your ability to explain forensic reasoning clearly.

Use the exercises only in the authorized lab environment and with the provided training materials. Record your process carefully, preserve your notes with the lab results, and adapt examples only within the authority and scope granted for your work.

1 Introduction to Digital Forensics

This chapter establishes the concepts and evidence practices that give hands-on investigation work its forensic meaning.

Digital devices record traces of human activity, system behavior, and security events. Digital forensics is the disciplined process of identifying, preserving, examining, analyzing, and reporting those traces so that conclusions can be explained and, when necessary, defended. The evidence may come from a computer, mobile device, removable drive, cloud service, network capture, or application log.

This book develops practical investigation skills, but technical skill alone is not enough. An analyst must also protect the original evidence, document each action, distinguish observations from conclusions, and recognize the limits of the available data. This chapter establishes that foundation, then compares digital forensics with the related disciplines that help organizations reduce and respond to risk.

Learning Objectives

- Define digital forensics and distinguish it from data recovery.
- Describe a controlled workflow from authorization and evidence gathering through imaging, examination, analysis, reporting, and case critique.
- Identify the authority, custody, integrity, method, and reporting practices that make evidence reviewable.
- Explain why evidence integrity, chain of custody, scope, and authorization matter.
- Distinguish a full bit-stream image from a typical file backup and explain why examiners use verified working copies.
- Classify common sources of digital evidence and identify their limitations.

- Compare digital investigations with vulnerability and risk management and incident response.
- Distinguish public- and private-sector investigation contexts while explaining the forensic practices they share.

1.1 Introduction to digital forensics

Digital forensics applies computer science and investigative procedures to identify, preserve, examine, analyze, and report digital evidence for a legal or organizational purpose. An investigation may seek to determine what happened, when it happened, which accounts or systems were involved, and what data was affected. The result is not merely a collection of files or screenshots; it is a reasoned account that another qualified examiner can review.

Evidence gains value from the process used to handle it. An examiner needs proper authority or authorization, a documented chain of custody, mathematical validation such as cryptographic hashes, validated tools and repeatable methods, and clear reporting that can support review or expert presentation. These practices make it possible to explain how the evidence was obtained, protected, examined, and interpreted.

Digital evidence may arise from user activity, applications, operating systems, and hardware.

Digital forensic work appears in criminal investigations, civil disputes, internal corporate inquiries, incident response, regulatory matters, and recovery from operational failures. The same artifact can answer different questions depending on the case. A USB drive, for example, might reveal evidence of file transfer, malware delivery, intellectual-property theft, or ordinary authorized work.

1.1.1 Why digital forensics matters

Digital evidence is often fragile. A system can overwrite logs, rotate temporary files, synchronize cloud data, or change timestamps during normal use. An investigator therefore needs to make sound preservation decisions before beginning detailed analysis.

Forensics also gives technical findings context. A security alert may show that suspicious activity occurred, while a forensic examination can correlate logs, files, network traffic, and metadata to explain the sequence of events and the scope of the impact.

A running example. Imagine that an organization detects unusual network traffic from an employee workstation shortly after a USB drive is connected. The security team must contain the potential threat, but the forensic questions are broader: Which USB device was used? What files were copied or executed? What network connections followed? Did the activity affect other systems? The techniques in this book help answer those questions while preserving a record of how each conclusion was reached.

1.1.2 Core goals of a digital forensic investigation

The central goals of an investigation are to locate relevant evidence, preserve it without unnecessary alteration, interpret it accurately, and communicate conclusions clearly. A strong examination separates what is directly observed from what is inferred.

For example, a browser-history record may directly show that a URL was stored in a particular user profile. It may support an inference that someone using that profile accessed the site, but it does not by itself prove the identity or intent of the person at the keyboard. This distinction is essential when reporting results.

1.1.3 Digital Forensics versus Data Recovery

Data recovery and digital forensics may use some of the same tools, but they answer different questions.

Activity	Primary question
Data recovery	Can we get the data back?
Digital forensics	Can we prove where the data came from, what it means, and whether it is reliable evidence?

Forensic work preserves the original media whenever possible and uses verified working copies for examination. Examiners may begin before they know whether

relevant evidence exists, so they must document the search process as well as the results. In the USB scenario, recovering a deleted document establishes that an artifact was recovered from the examined image; it does not prove who deleted it or why. The final product must therefore be explainable, repeatable, and defensible.

1.1.4 Types of digital evidence

Digital evidence includes both the contents of data and the metadata surrounding it. Common sources include filesystems, operating-system logs, browser artifacts, email, memory, network captures, mobile-device records, cloud audit logs, and application databases.

No single source tells the whole story. File timestamps may be ambiguous, logs may be incomplete, and user-controlled data can be misleading. Investigators build confidence by correlating independent artifacts and documenting both supporting and contradictory evidence.

For example, a data-leakage investigation may require evidence from a device, its operating system, and supporting network infrastructure before the sequence of events can be reconstructed [Xu et al., 2019].

1.2 Digital Investigation Workflow

A defensible investigation moves evidence through a controlled workflow. NIST guidance distinguishes identifying potential evidence sources, collecting data with minimal alteration, examining artifacts for review, and analyzing their meaning in relation to case questions [Kent et al., 2006]. For beginners, this book makes the practical handoffs explicit: authorization, evidence gathering and preservation, forensic acquisition, examination and analysis, reporting, and case critique.

Figure 1.1 shows the five-stage workflow. Hash values, notes, tool versions, timestamps, and custody records connect every stage. The work is iterative: an analytical finding may require additional authorized collection or a revised question.

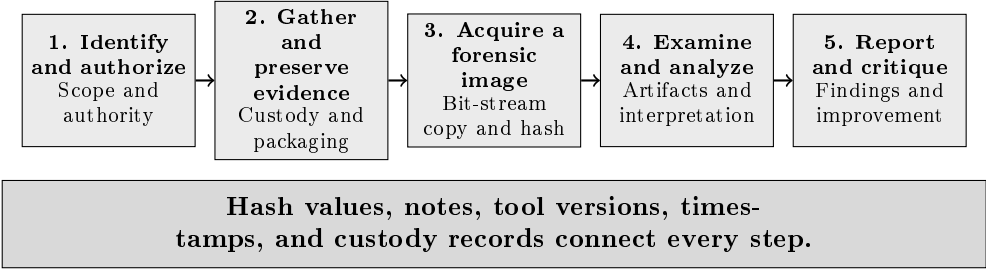


Figure 1.1: Digital investigation workflow from source to report. Controls span the process, and findings can require a return to an earlier stage.

1.2.1 Gathering Evidence and Initial Custody

Before collecting media, define the authorized question, scope, likely sources, and responsible people. Interview the responsible custodian or IT manager when appropriate to establish the device’s role, condition, and relevant time period. Complete the evidence form, obtain required signatures, place the item in a secure labeled container, document transport and every handoff, then control access in secure storage.

Evidence integrity means being able to show that the data examined is the same data that was collected, or that any changes are known and explained. Cryptographic hashes are commonly used to verify that a forensic image or exported file has not changed between collection and analysis [Kent et al., 2006, Scientific Working Group on Digital Evidence, 2023]. Chain of custody is the documented history of handling: who collected the item, when and where it was collected, its condition, storage, transfers, and examinations.

Figure 1.2 shows common physical-recordkeeping tools. A custody sheet records the item identity and each handoff; a tamper-evident security bag can carry the item information and seal identifier with the media. These tools support custody documentation but do not replace an authorization record, acquisition notes, or cryptographic integrity checks.

Figure 1.3 connects a package label to a custody record through the same case and item identifiers. Each handoff records both parties, the time and purpose, and their acknowledgments. A seal identifier and condition record help track the package; they do not replace verification of the digital contents.

[illegible]

(a) Blank chain-of-custody sheet

☐ SECURITY SEAL ☐ SECURITY SEAL ☐ SECURITY SEAL ☐ SECURITY SEAL
☐ SECURITY SEAL ☐ SECURITY SEAL ☐ SECURITY SEAL ☐ SECURITY SEAL

Caution: ATTEMPTS TO OPEN WILL INDICATE TAMPERING

ALERT

SECURITY BAG

Serial Number: 9724601

EVIDENCE

Station/Section/Unit/Dept: _____
 Case Number: _____
 Type of Offense: _____
 Description of Evidence: _____

Suspect: _____
 Victim: _____
 Date and Time of Recovery: _____
 Location of Recovery: _____
 Received by: _____

CHAIN OF CUSTODY

Released From _____	By _____	AMT / PMT
Received From _____	Time _____	
Released From _____	By _____	AMT / PMT
Date _____	Time _____	
Received From _____	By _____	AMT / PMT
Date _____	Time _____	

WARNING: TAMPER EVIDENT SECURITY PACKAGES. ONLY DESIGNED AND APPROVED TO OPEN ON-AIR OR BY DESIGNED OR AUTHORIZED.

STACY MORROW

CHIEF

www.stacy.com

1-800-268-7211

(b) Example tamper-evident security bag

Figure 1.2: Examples of physical evidence controls. Use the applicable organization or jurisdiction's approved forms and packaging procedures.

Controlled storage
Locker L-02
Access recorded

Acknowledged: B. Chen / C. Diaz

Figure 1.3: Illustrative evidence packaging and custody entries for a training USB drive. Later opening, resealing, and return to storage require additional records.

1.2.2 Acquisition: Forensic Images versus Backup Copies

A full bit-stream image captures the readable, addressable contents of a device sector by sector, including file-system structures and unallocated space. A sector is a unit of storage on a disk; unallocated space is storage that the file system currently considers available for reuse. Such space may retain deleted content, although recovery is not guaranteed. An image does not automatically capture inaccessible areas or data that has already been overwritten.

A typical file backup preserves selected known files and folders for restoration. It usually omits unallocated space, deleted-file records, and other low-level structures needed for forensic examination. The distinction matters in the USB scenario: a backup may contain visible documents, while a full image can also support examination of deleted files and remaining data.

Forensic image	Backup copy
Bit-by-bit acquisition of addressable storage; can retain file-system structures, unallocated space, deleted data, and fragments; validate with cryptographic hashes.	Selected known files for restoration; commonly omits deleted data, unallocated space, and fragments; useful operationally but insufficient by itself for a forensic examination.

Figure 1.4 visualizes the difference. A forensic image preserves the whole address space, including areas that may contain deleted data or fragments; a backup usually preserves only selected live files. The colored blocks represent data areas included in each copy, not the contents or evidentiary value of particular files.



Figure 1.4: Conceptual comparison of a bit-stream forensic image and a typical backup. A backup may serve restoration needs while omitting data areas needed for forensic examination.

Investigators preserve the original media and examine verified working copies of the image. Record the acquisition tool, settings, source, destination, time, and hash values so another examiner can understand and check the process. Tools

such as FTK Imager, EnCase, and dd support acquisition workflows; their use requires appropriate controls and verification.

A common first analysis exercise begins with a supplied image and uses a tool such as Autopsy to examine it. This separates the first analysis exercise from device acquisition while retaining the same requirements for documentation, integrity checks, and cautious interpretation.

1.2.3 Examination and Analysis

Examination processes the acquired image so artifacts can be identified, verified, and reviewed. Relevant sources may include operating-system artifacts and user profiles; application data, logs, configuration files, and metadata; deleted files and file fragments; and memory or other volatile artifacts when they were collected. Analysis interprets these artifacts in relation to the case question by correlating sources, testing explanations, constructing timelines, and recording limitations.

In the USB scenario, the examiner can analyze the image for file-system artifacts while correlating them with workstation logs and the network capture. A recovered document establishes that an artifact was found in the examined image; it does not by itself establish who used the drive, why the file was deleted, or the user's intent.

1.2.4 Reporting and Case Critique

A final report should state the authorized question and scope; identify evidence items and working images; record the tools, versions, settings, and procedures used; and present repeatable findings with their supporting artifacts. Address who, what, when, where, why, and how when the evidence permits. Separate conclusions from assumptions, uncertainties, and limitations. When useful to the intended audience, an examiner may also formalize artifacts and relationships using a structured representation such as STIX.

After reporting, critique the case to improve future performance. Ask whether the results were expected, what changed during the case, whether documentation is complete enough for review, and what feedback came from the requesting source. Record new problems, tool limitations, or training gaps, and update procedures, checklists, or lab practices when warranted. This review supports future work; it

does not silently change the original record. Corrections to a finding must state what changed and why.

1.3 Digital Forensics and Related Disciplines

Digital forensics works alongside disciplines that prevent harm, respond to active incidents, and reconstruct events. Their goals overlap, but each uses evidence for a different primary purpose.

Figure 1.5 presents these connected activities as an investigations triad.

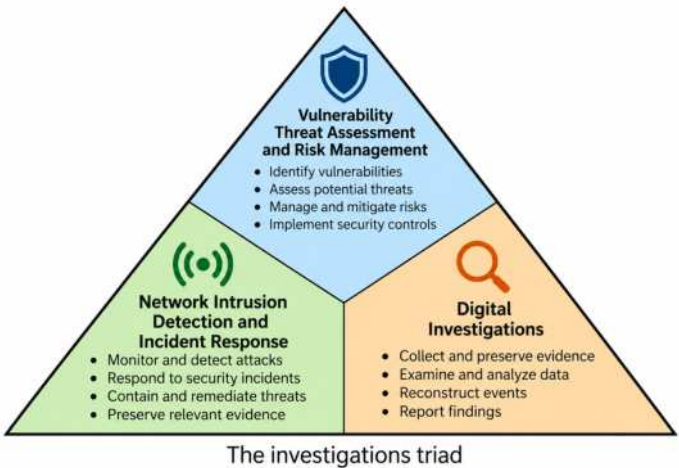


Figure 1.5: The investigations triad: risk management, incident response, and digital investigations support different stages of security work.

Vulnerability, threat assessment, and risk management identifies vulnerabilities, assesses threats, and reduces risk before an incident occurs. **Network intrusion detection and incident response** detects attacks, contains and remediates incidents, and preserves relevant evidence. **Digital investigations** collects, preserves, examines, and analyzes digital evidence to reconstruct and explain what happened.

Discipline	Primary purpose	Relationship to forensic evidence
Vulnerability, threat, and risk management	Identify vulnerabilities, assess threats, and reduce risk before an incident occurs.	Asset inventories, configurations, and risk records help define likely evidence sources and prevention priorities.
Network intrusion detection and incident response	Detect attacks, contain and remediate incidents, and restore operations.	Alerts, logs, and captures provide leads; responders preserve relevant evidence while acting quickly to limit harm.
Digital investigations	Collect, preserve, examine, and analyze digital evidence to reconstruct and explain what happened.	The examiner produces reliable, traceable findings that can support organizational decisions, remediation, or legal review.

1.3.1 Public- and Private-Sector Digital Investigations

Digital investigations occur in different organizational settings. The authority for the work, intended audience, and possible outcome differ, but both settings require careful preservation, documentation, custody records, validated methods, and reproducible findings.

Context	Typical purpose, examples, and outcomes
Public sector	Government-led criminal or civil investigations. The work follows the applicable legal process and evidentiary requirements; findings may support prosecution, court proceedings, or another government decision.
Private sector	Internal policy, compliance, or incident investigations. Examples include harassment, data falsification, embezzlement, sabotage, and espionage; findings may support HR action, litigation, remediation, or a decision to refer the matter to an appropriate authority.

In the USB-and-network scenario, a public-sector investigation might examine whether the USB activity relates to an alleged offense. A private-sector investigation might examine whether it violated organizational policy or exposed business information. The training lab does not establish a legal conclusion; it develops the same careful practices needed to preserve, examine, and explain evidence in either setting.

In the USB-and-network scenario, risk management might identify the need to control removable media, intrusion detection might alert the organization to unusual traffic, and incident response might isolate the workstation. A digital investigation then examines the USB image, logs, and network evidence to determine what those events show and how reliably they support a conclusion. Speed matters during an incident, but responders should record the system state, collection time and timezone, people involved, and tools used. When possible, they should preserve original data and analyze verified copies [Kent et al., 2006, Nelson et al., 2025].

1.4 Summary

Digital forensics applies computer science and investigative procedures to identify, preserve, examine, analyze, and report digital evidence for a legal or organizational purpose. Evidence is reviewable when authority, custody, integrity checks, validated repeatable methods, and clear reporting are maintained. The NIST lifecycle distinguishes identification, collection, examination, and analysis; this book’s workflow makes authorization, evidence gathering and preservation, forensic acquisition, examination and analysis, reporting, and case critique explicit. A

full bit-stream image retains storage structures that a typical file backup omits, supporting examination of deleted content and unallocated space. Recovering data is only part of forensic work: findings need traceable methods, supporting artifacts, and explicit limitations. Vulnerability and risk management reduces risk before incidents, incident response contains active harm, and digital investigations reconstruct events from reliable evidence in public- or private-sector contexts.

Chapter Review

Review Questions

1. What is the difference between a direct forensic observation and an inference?
2. What authority, custody, integrity, method, and reporting practices make digital evidence reviewable?
3. What actions and controls distinguish the five stages of the digital investigation workflow?
4. How do vulnerability and risk management, incident response, and digital investigations differ in their primary purposes?
5. How do public- and private-sector digital investigations differ in authority and possible outcomes, and what forensic practices do they share?
6. Why are hashes and chain-of-custody records important when handling evidence?
7. How does a full bit-stream image differ from a typical file backup, and why does that matter when investigating deleted files?
8. Name three common sources of digital evidence and one limitation of each.
9. In the USB scenario, what does recovering a deleted document establish, and what further questions belong to the forensic examination?
10. Why can a running system require different collection decisions than powered-off media?
11. What should a final report distinguish between, and how should a case critique improve later investigations without changing the original record?

Discussion Questions

1. When should a responder prioritize volatile-data collection over immediate shutdown? What risks must be documented?
2. How should a responder balance rapid containment with the need to preserve evidence for a later investigation?
3. How might the same USB-and-network evidence be used differently in a public-sector and a private-sector investigation?
4. Which workflow records should be maintained across every stage, and how would they help another examiner review the case?

Hands-On Exercise

Exercise. Using the USB-and-network scenario in this chapter, create a one-page investigation workflow plan. State the authorized question; list likely evidence sources; describe initial custody, secure storage, and integrity verification; explain why a forensic image is required instead of only a backup; identify artifacts to examine; and name the report contents and one case-critique improvement. Distinguish two observations from two possible inferences. Do not collect data from systems without authorization.

2 Your First Investigation: USB Image Analysis with Autopsy

Learning Objectives

- Create and configure an Autopsy case for a forensic image.
- Prepare a supplied image, record baseline hashes, and browse its file-system view in Autopsy.
- Identify deleted content and explain the limits of recovery results.
- Tag relevant evidence and generate a focused forensic report.
- Relate Autopsy findings to an investigative question.

USB forensics plays a vital role in contemporary digital investigations because portable storage devices often contain user documents, deleted files, and traces of activity that can become important evidence. This chapter focuses on examining a provided forensic image of a USB drive with **Autopsy**, an open-source forensic platform for browsing files, recovering deleted content, and generating reports.

Rather than teaching full acquisition of the original USB device, this chapter assumes that a forensic image has already been created and concentrates on the analysis workflow that follows. By the end of the chapter, readers will have practiced preparing the image, verifying its integrity locally, loading it into Autopsy, recovering deleted files, and exporting a report that documents their findings.

Chapter 1 introduced evidence handling, forensic images, and integrity checks. Here you will apply those ideas to a small investigation: What files can be recovered from the supplied USB image, and what do those files actually establish? Record observations separately from conclusions about who used the drive or why a file was deleted.

This first lab uses Windows and Autopsy's graphical interface; it does not require

the later Linux chapters. A forensic image is a file containing captured storage data, not a photograph or a folder of copied documents. The supplied image lets you practice examination before learning to acquire a device yourself in Chapter 8.

2.1 Investigating a USB Disk Image with Autopsy

This section walks through a practical Autopsy lab workflow for a provided USB forensic image. The preparation stage focuses on installing the tool, obtaining the sample archive, extracting the actual image file, and documenting baseline hash values before the image is loaded into the case.

2.1.1 Introduction to Autopsy

“**Autopsy**” is a powerful and versatile open-source digital forensics platform that simplifies analyzing digital evidence. It provides a user-friendly graphical interface, making it accessible to novice and experienced investigators. Autopsy offers a wide range of features, including file system analysis, keyword search, data carving, timeline creation, and report generation. It supports various file systems and can analyze evidence from different types of hard drives, USB drives, and memory cards.

2.1.2 Preparing the Autopsy environment and evidence image

Before starting the analysis, prepare both the software environment and the sample image that will be examined in the case.

Installing Autopsy in Windows. Consult the official Autopsy download page. Choose the latest stable release and select the 64-bit Windows installer if your system supports it.

What to notice in Figure 2.1.

Latest stable release Download the current stable version rather than an out-dated or experimental build.



Figure 2.1: Autopsy download page (source: Autopsy official site)

Windows installer Choose the installer that matches the system architecture, which will usually be 64-bit on modern systems.

1. After downloading the Autopsy installer, locate and “**double-click**” the file to initiate the installation process.
2. If prompted by User Account Control, grant permission by clicking “**Yes.**” Proceed through the installation wizard, typically accepting default settings unless customization is required.
3. Once the installation completes, click “**Finish.**” You might be asked if you want to associate **.aut** files with Autopsy, which is recommended.
4. You can now launch Autopsy from the newly created shortcut on your desktop or in your Start menu.

Downloading the sample archive. To conduct the analysis in this chapter, download the sample archive **120M.7z** from the companion repository at GitHub download link.

What to notice in Figure 2.2.

Archive format The downloaded item is the compressed archive **120M.7z**, not the forensic image that Autopsy will open directly.

Next action The archive must be extracted before the data source can be loaded into Autopsy.

Extracting the archive. The downloaded file **120M.7z** is compressed for distribution. After extraction, the actual forensic image used in this lab is **usb_-**

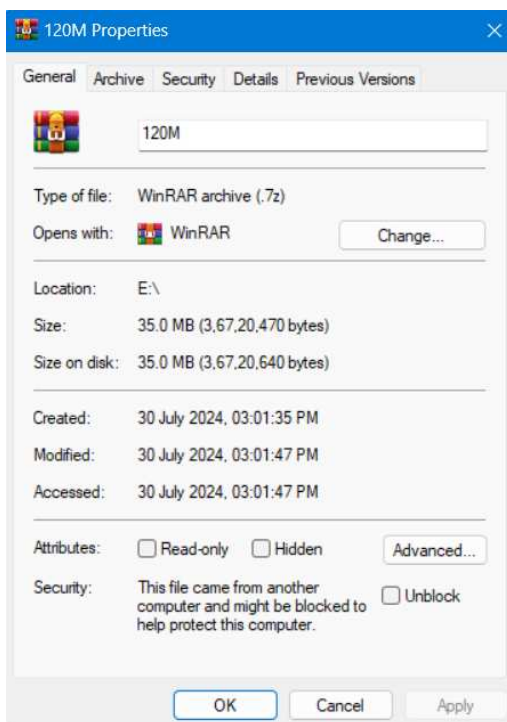


Figure 2.2: File property of USB image

`fat_carving.001`.

For Windows, “**7-Zip**” is a recommended, free, and open-source file archiver that can handle 7z files. If you don’t have it already, download and install it from the official 7-Zip website. Once **7-Zip** is installed, follow these steps to extract the USB image:

1. **Locate the Archive:** Find the downloaded “120M.7z” file in your Downloads folder or wherever you saved it.
2. **Right-Click and Extract:** Right-click on the “120M.7z” file.
3. **Choose Extraction Option:** From the 7-Zip context menu, you have several options:
 - **Extract Here:** This will extract the contents directly into the same folder as the archive.
 - **Extract to 120M:** This will create a new folder named “**120M**” and extract the contents there.
 - **Extract Files...:** This will open a dialog box where you can customize the extraction location and other options.

Choose the option that suits your preference. Once extraction is complete, confirm that the file you will later load into Autopsy is `usb_fat_carving.001`.

Understanding a hash. A cryptographic hash is a fixed-length value calculated from a file’s contents. Recalculate it with the same algorithm to check whether a working copy still matches the recorded baseline. A matching hash supports integrity; it does not establish who created the contents or whether they are accurate. Windows provides `certutil` for this calculation; the `-hashfile` option takes a filename and the hash algorithm. Open Command Prompt in the folder containing the extracted image before running the commands below.

Verifying the image locally with `certutil`. Before loading the image into Autopsy, record baseline hash values for `usb_fat_carving.001`. This is a local integrity step: it lets the investigator document the file’s state without uploading evidence-derived material to a third-party website. If your instructor or lab bundle provides reference hash values, compare your results against them. Otherwise, record these values in your notes so you can compare later copies of the complete image against the same baseline. Extracted documents have their own contents and hashes; do not compare their hashes to the image hash.

Figure 2.3 shows the MD5 calculation, and Figure 2.4 shows the corresponding

SHA-256 calculation for the same image file.

```
C:> certutil -hashfile usb_fat_carving.001 MD5
MD5 hash of usb_fat_carving.001:
ba4a1d0ba49f4a6667b00a3b3e85e604
CertUtil: -hashfile command completed successfully.
```

Figure 2.3: Calculating the MD5 hash of the extracted USB image with certutil

```
C:> certutil -hashfile usb_fat_carving.001 SHA256
SHA256 hash of usb_fat_carving.001:
9bfeb5634ade30764f0e581a4686930f7fab0472378595b789b0cdb248c91d9
CertUtil: -hashfile command completed successfully.
```

Figure 2.4: Calculating the SHA-256 hash of the extracted USB image with certutil

MD5 command Run `certutil -hashfile usb_fat_carving.001 MD5` to document an MD5 digest for the image.

SHA-256 command Run `certutil -hashfile usb_fat_carving.001 SHA256` to document a stronger modern digest for the same file.

What to record The important output is the hash string itself and the fact that **CertUtil** reports the command completed successfully.

2.2 Analyzing a USB image with Autopsy

In this section, the prepared image is loaded into an Autopsy case so the examiner can inspect its file system, locate deleted content, and document the findings.

2.2.1 Creating a New Case

After launching Autopsy, the first step in your investigation is to create a new case to organize and manage the analysis of your USB image.

1. **Launch Autopsy:** Double-click the Autopsy icon on your desktop or find it in your Start menu.

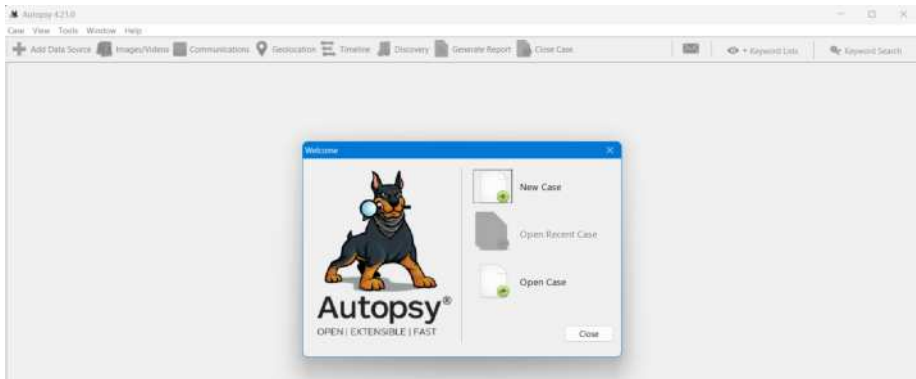


Figure 2.5: Autopsy Home screen

2. **Create New Case:** On the Autopsy welcome screen, click the “**New Case**” button (Figure 2.5).
3. **Case Information:** You’ll be prompted to enter the following details:
 - **Case Name:** Choose a descriptive name for your case.
 - **Base Directory:** Select a directory where Autopsy will store the case files.
 - **Case Type:** Choose “**Single-User**” unless you are collaborating with other investigators.
 - **Case Number:** Enter a unique case number if your organization has a standardized format.
 - **Examiner Name:** Enter your name or the name of the investigator leading the case.
4. **Finish:** Click the “**Finish**” button to create the new case.

Figure 2.6 illustrates the “**New Case**” dialog box, showing the fields where you enter the necessary information.

What to notice in Figure 2.6.

Case name Use a descriptive case name so the lab output can be distinguished from other Autopsy cases.

Base directory This is where Autopsy will store its case database, reports, and supporting files.

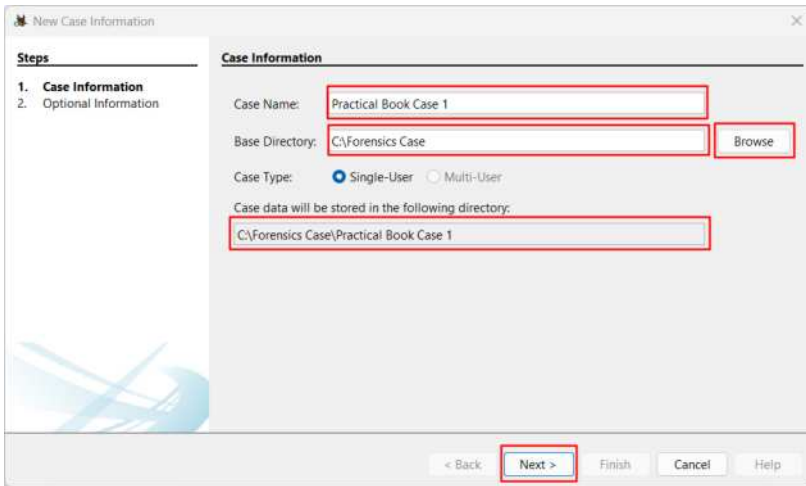


Figure 2.6: Creating and entering details of the new case in the autopsy tool

Single-user case This is the appropriate choice for a solo lab workflow unless the case is being shared among multiple examiners.

Optional case information. After entering the basic case details, Autopsy provides an additional screen for optional metadata such as case number, examiner contact information, organization, and notes.

1. **Case Number:** Choose a unique identifier for this case.
2. **Examiner Name:** Enter the name of the examiner for your case.
3. **Phone:** Enter the examiner’s phone number.
4. **Email:** Enter the examiner’s email address.
5. **Notes:** Add a brief note for context (e.g. “This is the first case,”).
6. **Organization:** The "Not Specified" option is selected, but you could choose a specific organization if applicable.

After filling in the necessary details, click the “**Finish**” button to complete the case creation process and move on to adding the data source (USB image) for analysis.

Figure 2.7 shows the optional case-information screen where those supplemental details can be recorded.

Steps

1. Case Information
2. **Optional Information**

Optional Information

Case

Number: 120M__01

Examiner

Name: Frank Xu

Phone: 000111222

Email: fmaikw.xu@gmail.com

Notes: This is the first case

Organization

Organization analysis is being done for: Not Specified Manage Organizations

< Back Next > **Finish** Cancel Help

Figure 2.7: Adding optional case information

2.2.2 Adding the USB image as a data source

After the case is created, the next task is to add the forensic image as the data source that Autopsy will analyze. Because this lab uses a previously prepared image file rather than a live disk, the correct choice is “**Disk Image or VM File**.”

Figure 2.8 illustrates the “**Select Data Source Type**” dialog in Autopsy, highlighting the “**Disk Image or VM File**” option.

What to notice in Figure 2.8.

Disk Image or VM File This is the correct option because the lab works from the extracted forensic image `usb_fat_carving.001`.

What not to load Do not select the compressed archive `120M.7z`; Autopsy should open the extracted image file instead.

After choosing the source type, locate and load the extracted USB image file.

1. In the “**Select File**” dialog box (Figure 2.9), navigate to the directory where you saved the extracted image `usb_fat_carving.001`. Double-click the file to select it.
2. The selected file name will appear in the “**File Name**” field at the bottom of the dialog box. Double-check that you have selected the correct file.

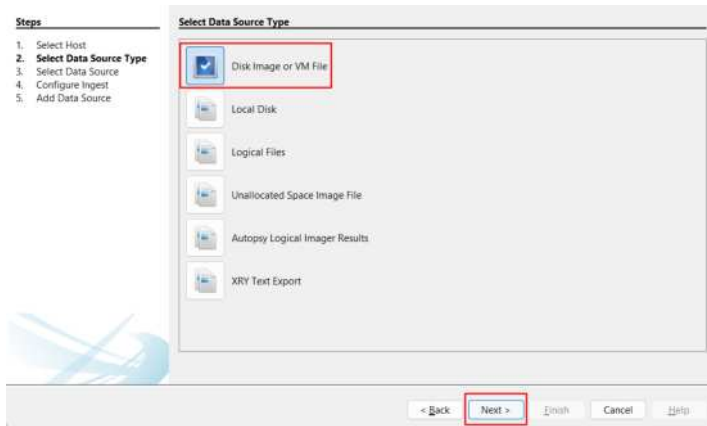


Figure 2.8: Selecting the data source type

3. Click the “**Open**” button to load the USB image into Autopsy.

Figure 2.9 illustrates the process of selecting the USB image file in the “Select File” dialog box.

In the next step of “**Selecting Data Source**” (Figure 2.10), review the image path and case settings before clicking “**Next**” to begin the analysis.

What to notice in Figure 2.10.

Image path Confirm that the selected file is `usb_fat_carving.001`, not the compressed archive.

Time zone Use the correct time zone for the case because Autopsy relies on that setting when presenting timestamps during analysis.

Sector size Keep the default sector size unless the image source is known to require a different value.

Success indicator After the data source is accepted, Autopsy proceeds with ingest and the image appears within the case tree for browsing.

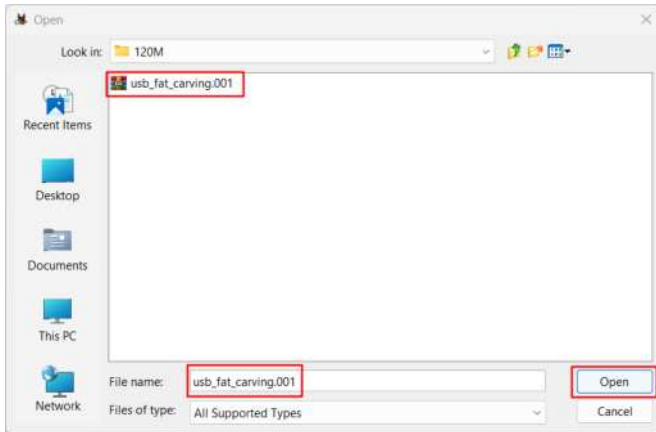


Figure 2.9: Selecting the disk image file

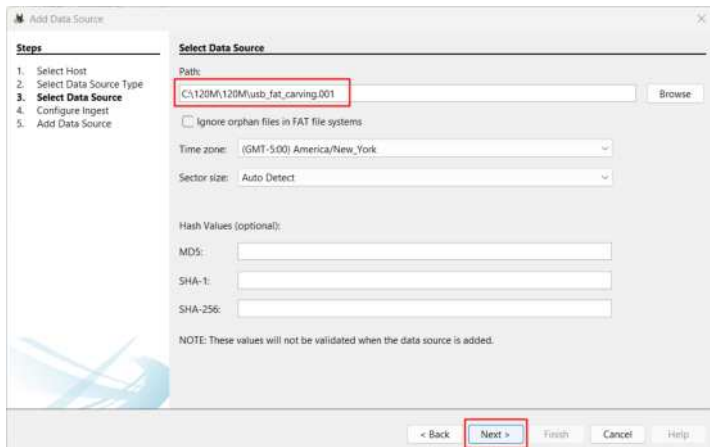


Figure 2.10: Select and add the data source

2.3 Recovering the deleted content from the USB image

In the course of a digital forensic investigation, deleted files often hold valuable evidence. A file system is the set of structures that organizes files and records where their data is stored. Deleting a file can mark its storage as available for reuse without immediately erasing all of its contents. Deleted files aren't always truly gone. While they may be removed from the file system's directory, their data often remains on the storage media until overwritten. This allows forensic investigators to potentially recover the files, revealing hidden or intentionally erased information crucial to the case. This section explores techniques for identifying and recovering such files from the USB image we've been working with.

2.3.1 Finding the files deleted from the USB

Autopsy provides a visual indication of deleted files within the file list view. As shown in Figure 2.11, files marked with a red “X” icon are identified as deleted. These files may be recoverable depending on various factors such as the file system, the extent of data overwriting, and the effectiveness of Autopsy's recovery modules.

In Figure 2.11, you can observe several deleted files, including Word documents (.doc and .docx), Excel spreadsheets (.xls), and PowerPoint presentations (.pptx). Identifying deleted files is crucial in forensic investigations, as they may contain intentionally removed or accidentally lost evidence, potentially revealing hidden information or deleted communications relevant to the case.

2.3.2 Tagging deleted files for review

Autopsy's tagging feature allows investigators to mark files of interest within the USB image. As shown in Figure 2.12, right-clicking on a file reveals a context menu with various options, including “**Add File Tag.**” Tagging helps organize significant files and makes them easier to collect later for reporting or extraction.

What to notice in Figure 2.12.

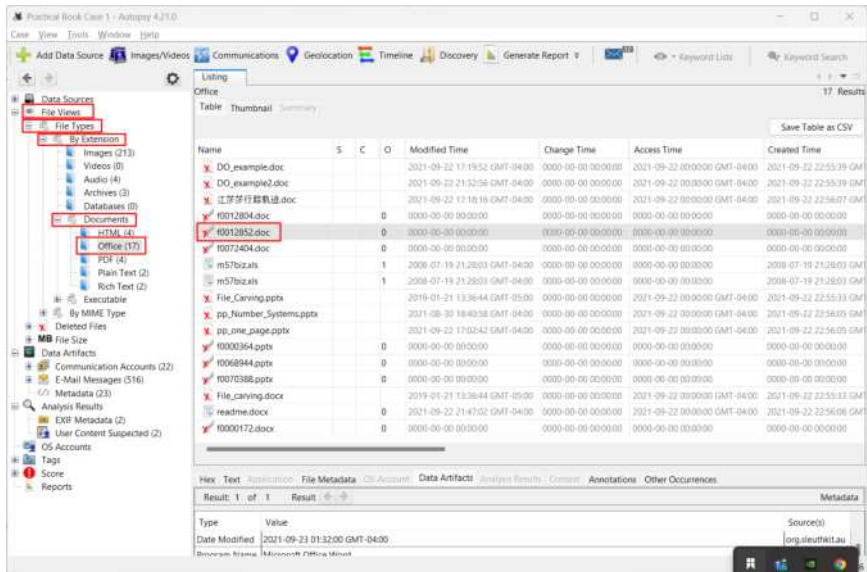


Figure 2.11: Autopsy window displaying deleted files

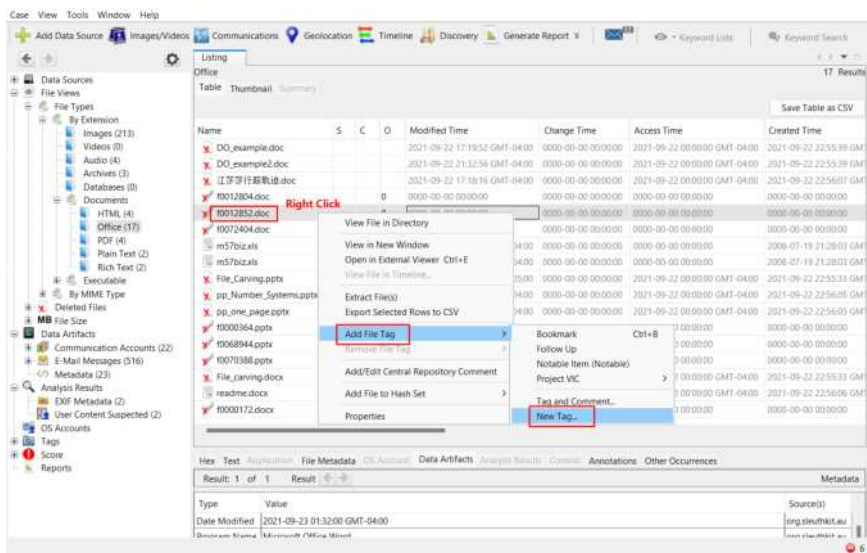


Figure 2.12: Applying tags to a file for reference

Context menu action “Add File Tag” is the command used to mark a file for later review.

Why this matters Tagged items can be filtered, exported, and included more easily in later reporting steps.

Naming the tag. After creating a tag in Autopsy, give it a meaningful name that reflects its purpose or the type of files associated with it. This helps you identify and filter tagged items during analysis and reporting.

- Right-click on the tag in the “Tags” section of the left sidebar.
- Select “Rename Tag” from the context menu.
- Enter a new name for the tag in the dialog box that appears (Figure 2.13).
- Click “OK” to save the changes.

Figure 2.13 shows the dialog box where you can customize the tag name. Using descriptive tag names, you can create a well-organized and easily navigable structure within your Autopsy case, making locating and analyzing relevant evidence easier.

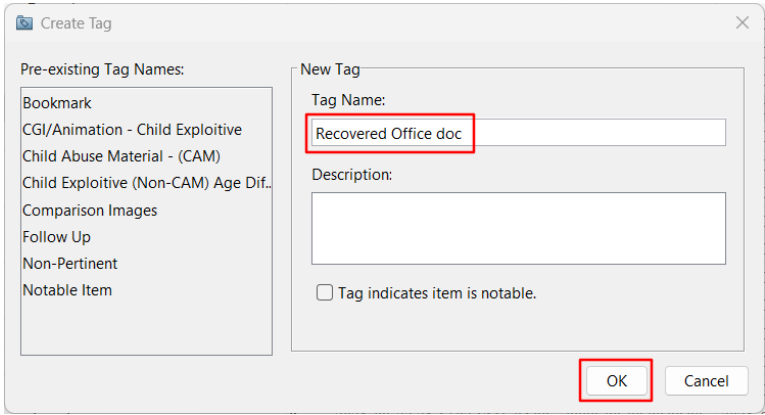


Figure 2.13: Enter the tag name to create it

Applying tags to deleted files. After identifying deleted files within Autopsy, it is often helpful to mark them for further investigation. In Figure 2.14, two files, `f0012852.doc` and `f0072494.doc`, are highlighted. The red “X” indicates that these are deleted files, while the yellow tagging indicator shows they have

been marked for later review.

Tagging allows you to:

- Group related files or artifacts together for easier reference and analysis.
- Focus your attention on items that seem particularly relevant to the investigation.
- Easily include tagged files in your forensic reports.

To tag a file in Autopsy, right-click on the file and select “Tag File.” You can then assign a tag from the list of existing tags or create a new one.

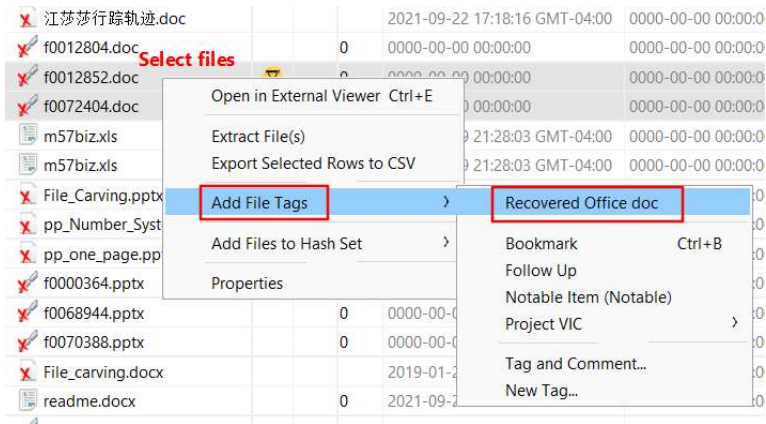


Figure 2.14: Adding file tag

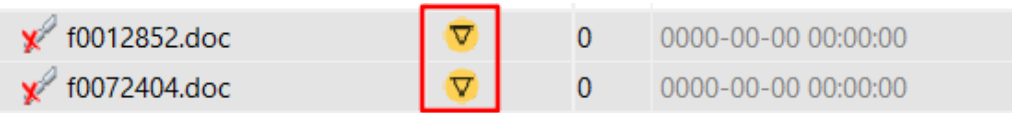


Figure 2.15: Tagged files

What to notice in Figures 2.14 and 2.15.

- Red X** The file is deleted in the file system view.
- Tag indicator** The file has been marked for later review, extraction, or reporting.

Workflow implication Once a file is tagged, it becomes easier to collect related artifacts as a group.

2.3.3 Recovering and extracting the tagged files

After identifying and tagging relevant files, the next step is to recover and extract them for further analysis outside of the Autopsy environment. This process involves exporting the files to a designated location on your system. To recover and extract a tagged file:

- Navigate to the file you want to extract in the Autopsy Tree Viewer or Results View.
- Right-click on the file and select “Extract File(s)” from the context menu.
- In the dialog box that appears, select the destination folder where you want to save the extracted file.
- Click “OK” to initiate the extraction process. Autopsy will save a copy of the file to the specified destination.

You can extract multiple tagged files simultaneously by selecting them and then choosing "Extract File(s)." Alternatively, you can export all tagged files by right-clicking on the tag itself in the Tags panel and selecting "Export Files with Tag."

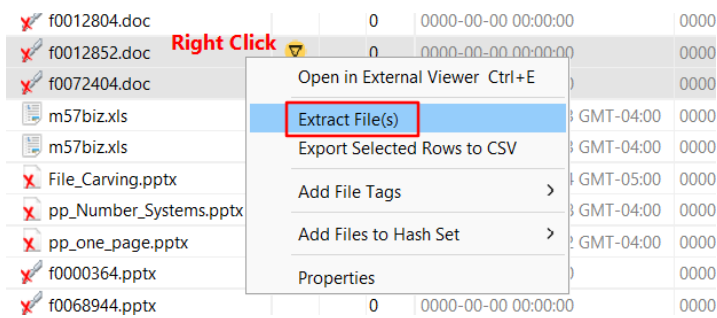


Figure 2.16: Extracting the recovered files

What to notice in Figure 2.16.

Extract File(s) This action exports a copy of the selected evidence item outside

the Autopsy case for later review.

Destination choice The save location should be documented because extracted evidence copies become part of the working record.

The recovered documents can then be stored at the desired location on the computer.

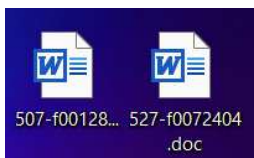


Figure 2.17: Recovered files

What to notice in Figure 2.17. The exported files now exist outside the Autopsy case and can be opened, reviewed, or preserved as examiner working copies without changing the original image.

2.4 Generating the final forensic report

After a comprehensive investigation of the USB image, the culmination of your efforts lies in the generation of a clear, concise, and informative forensic report. This report serves as a formal documentation of the entire investigative process, including the findings, methodologies used, and any relevant conclusions drawn. This section will guide you through the process of generating a comprehensive forensic report using Autopsy, ensuring that your findings are well-documented and presented clearly and professionally.

Generating the report. After completing your analysis in Autopsy, the final step is to generate a report summarizing your findings. Autopsy provides several report formats, including HTML, Excel, and body files. The choice of format depends on your needs, but HTML is a practical default for review and sharing.

- In the Autopsy menu bar, click on the “**Generate Report**” button (shown in Figure 2.18).
- In the “**Generate Report**” dialog box, select “**HTML Report**” and optionally configure the header and footer of the report (Figure 2.18). You can leave

these fields blank if you do not need to add any specific information.

- Autopsy offers additional report modules that you can enable, such as an Excel report, a list of tagged files, or a data source summary report.
- Click the “**Next**” button to continue and then click the “**Finish**” button to generate the report.

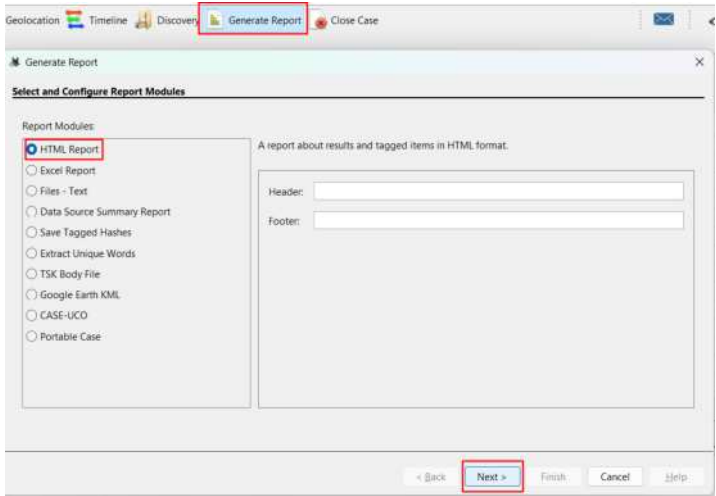


Figure 2.18: Configuring an HTML forensic report

What to notice in Figure 2.18.

HTML Report This is the main output option used in the lab because it is easy to browse and review.

Optional modules Tagged files and summary modules can make the final report more useful by surfacing the evidence items highlighted during analysis.

Visible success state After finishing the wizard, Autopsy writes the report to the output directory for later review.

Autopsy will then generate the selected reports and save them in the designated output directory. The HTML report can be opened in any web browser, providing a structured and visually appealing overview of your forensic findings.

Reviewing the generated report. Once Autopsy has finished generating the forensic report, open it and review the high-value sections first. Start with the case summary and tagged files so you can confirm the report identifies the image,

the examiner, and the recovered evidence items you deliberately marked during analysis.

Figure 2.19 shows the HTML report generated by Autopsy for the case “**Practical_Book_Case_1**.” The report is divided into two main sections:

1. **Report Navigation:** This section on the left in Figure 2.19 provides a list of the different categories of information contained in the report, including:
 - **Case Summary:** Provides an overview of the case, including the case number, examiner name, and notes.
 - **Accounts Email:** Lists any email accounts found in the image.
 - **Data Source Usage:** Shows how much of the image has been analyzed.
 - **E-Mail Messages:** Lists all the email messages found in the image.
 - **EXIF Metadata:** Lists any EXIF metadata found in image files.
 - **Metadata:** Lists other metadata found in the image, such as file creation and modification times.
 - **Tagged Files:** Lists any files that have been tagged by the examiner.
 - **Tagged Images:** Lists any images that have been tagged by the examiner.
 - **Tagged Results:** Lists any results of analysis modules that have been tagged by the examiner.
 - **User Content Suspected:** Lists any files that Autopsy suspects may contain user-created content.
2. **Main Content Area:** This section on the right in Figure 2.19 displays the details of the selected category. In the image, the Case Summary is displayed, showing the case name, case number, examiner name, and notes. The image information shows the name of the image file.

What to inspect first in Figure 2.19.

Case Summary Confirm the case identifiers, examiner details, and image name are correct.

Tagged Files Review this section early because it usually surfaces the most relevant evidence items from the lab workflow.

Metadata and usage views Use these sections when you need supporting details

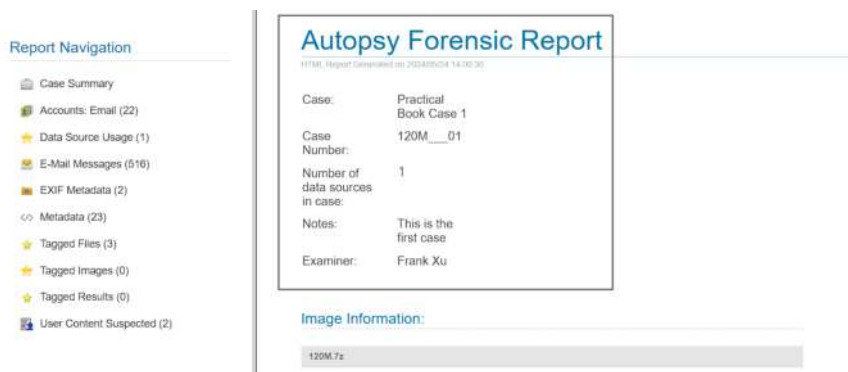


Figure 2.19: Overview of the generated report

such as timestamps, file properties, or broader context for the recovered artifacts.

2.5 Summary

This chapter used a provided USB forensic image to demonstrate a practical Autopsy analysis workflow. We applied the evidence practices introduced in Chapter 1 to a first investigation, then prepared the lab environment by installing Autopsy, downloading the archive `120M.7z`, extracting the image `usb_fat_carving.001`, and documenting baseline MD5 and SHA-256 hashes locally with `certutil`.

We then created a new Autopsy case, added the extracted image as the data source, and reviewed how deleted files are identified, tagged, and exported for closer analysis. The chapter concluded with report generation, showing how Autopsy organizes the final findings into a browsable HTML report that records the case context, tagged evidence, and supporting metadata. Together, these steps reinforce the core forensic habits of preserving the original evidence, working from the image, documenting integrity checks, and explaining findings clearly.

Chapter Review

Review Questions

1. Why should an examiner work from a forensic image rather than the original USB drive?
2. What does a baseline image hash let you check, and why should an extracted document not be compared with that image hash?
3. What is the value of recording case information in Autopsy?
4. How can tagged files improve a final report?
5. What does recovery of a deleted file establish, and what may it not establish?
6. Why should an Autopsy finding be corroborated with other artifacts when possible?

Discussion Questions

1. How can an examiner avoid allowing an automated artifact label to become an unsupported conclusion?
2. What criteria should determine whether a recovered file belongs in a report?

Hands-On Exercise

Exercise. Reopen the USB image used in this chapter in Autopsy. Retain the image hash and case details with your notes. Locate one allocated artifact and one deleted or unallocated artifact, tag both, and generate a report containing only those tagged items. For each item, write one observation and one limitation.

3 Number Systems for Digital Forensics

Learning Objectives

- Convert values among binary, decimal, hexadecimal, and octal representations.
- Interpret ASCII and Unicode values found in forensic artifacts.
- Convert UNIX timestamps to human-readable time and identify timezone considerations.
- Explain how hash values support integrity verification.
- Recognize how byte order affects multi-byte values.

Digital forensics depends on translating machine representations into defensible human interpretation. Investigators routinely encounter values stored as binary, hexadecimal, encoded text, UNIX timestamps, hash digests, and multi-byte fields inside low-level artifacts. To interpret those values correctly, they need to understand both what the representations mean and how to convert them quickly and accurately.

This chapter introduces the foundational representations that appear when reading command output, hex dumps, timestamps, file structures, and memory artifacts. We begin with number systems such as binary (base-2), decimal (base-10), octal (base-8), and hexadecimal (base-16), then connect those ideas to ASCII and Unicode text encoding, UNIX time, hash values, and byte order.

Mastering these representations is not just theoretical. It helps analysts explain what raw bytes mean, verify whether a value has been interpreted correctly, and move between machine-oriented and human-readable views of the same evidence. The chapter is organized around four practical threads: converting between number systems, translating between characters and their encoded values,

interpreting time values stored as numbers, and understanding how systems store and verify data at a low level. The following sections use Linux commands and Python examples to show how these ideas appear in day-to-day investigations.

This chapter applies the evidence-interpretation principles introduced in Chapter 1. Later chapters rely on these foundations when they examine file-system metadata, calculate and verify hashes, reconstruct timelines, and interpret network and storage artifacts.

3.1 Number system conversions in digital forensics

In digital forensics, converting between different number systems is a fundamental skill. Investigators frequently encounter data represented in binary or hexadecimal formats, while human interpretation often relies on the familiar decimal system.

This section will delve into the practical techniques for performing these conversions using both Linux command-line tools and Python scripting, providing forensic analysts with the means to efficiently translate and analyze digital evidence.

3.1.1 Linux-Based Number System Conversions

The Linux shell is often the fastest place to test an interpretation while examining logs, hex dumps, or memory values. The commands in this subsection help an investigator move quickly between the number systems most commonly encountered in forensic work.

Binary to decimal. Figure 3.1 summarizes the positional-weight method behind binary conversion, where each bit contributes a power of 2.

For example, the binary number ‘1011’ equals decimal 11 because $1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 = 8 + 0 + 2 + 1$. Figure 3.2 shows the shell shortcut `echo $((2#101010))`, which is useful when the analyst wants an immediate decimal interpretation of a binary value.

The shell evaluates the arithmetic expression first and then `echo` prints the result.

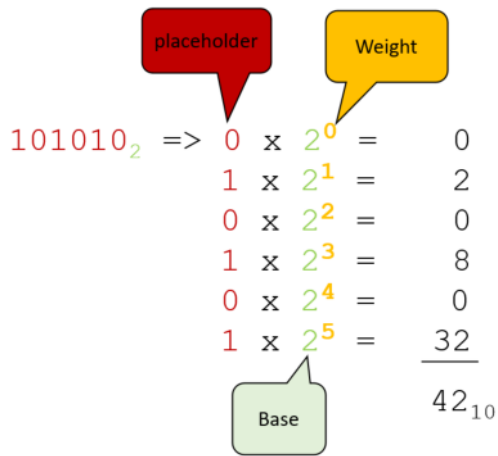


Figure 3.1: Illustration of binary to decimal conversion



Figure 3.2: Converting a binary number to its decimal equivalent

The output ‘42’ confirms that the binary value ‘101010’ corresponds to decimal 42. The important syntax is:

`$(...)` The shell arithmetic-expansion syntax.
`2#` The base prefix telling the shell to read the following digits as binary.
`echo` Prints the converted value so it can be compared with evidence.

Hexadecimal to decimal. Hexadecimal values appear constantly in forensic tools because they compactly represent binary bytes. Figure 3.3 shows the place-value method behind the conversion.

$$\begin{array}{rclclcl} \text{AB2}_{16} & \Rightarrow & (?)_{10} & & & \\ 2 \times 16^0 & = & 2 \times 1 & = & 2 & \\ \text{B} \times 16^1 & = & 11 \times 16 & = & 176 & \\ \text{A} \times 16^2 & = & 10 \times 256 & = & 2560 & \\ & & & & \hline & & & & 2738_{10} \end{array}$$

Figure 3.3: Illustration of hexadecimal to decimal conversion

For the hexadecimal value ‘AB2’, the decimal calculation is $10 \times 16^2 + 11 \times 16^1 + 2 \times 16^0 = 2560 + 176 + 2 = 2738$. Figure 3.4 demonstrates two shell commands that perform the same conversion.

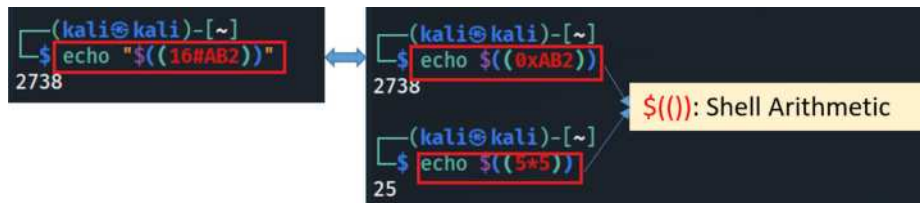


Figure 3.4: Demonstrating hexadecimal to decimal conversion

Using shell arithmetic. The command `echo ${16#AB2}` tells the shell to interpret ‘AB2’ as a base-16 number and print its decimal value. An equivalent form, `echo ${0xAB2}`, uses the more familiar hexadecimal prefix seen in many tools and programming languages. In each case, the key output to notice is ‘2738’. The forms differ only in notation: `16#AB2` is an explicit base prefix, while `0xAB2` is the more familiar hexadecimal prefix seen in many tools and

programming languages.

Decimal to binary. Investigators sometimes need to move the other way, especially when checking bit flags or permissions. Figure 3.5 shows the repeated-division-by-2 method that produces binary digits from a decimal number.

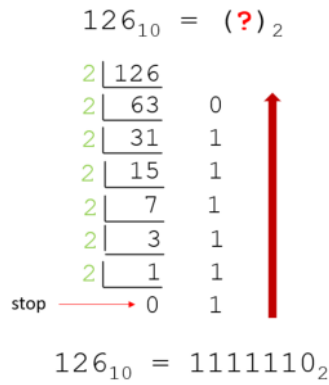


Figure 3.5: Illustration of decimal to binary conversion

The shell example in Figure 3.6 uses `echo "obase=2; 126" | bc` to produce the binary value '1111110'.

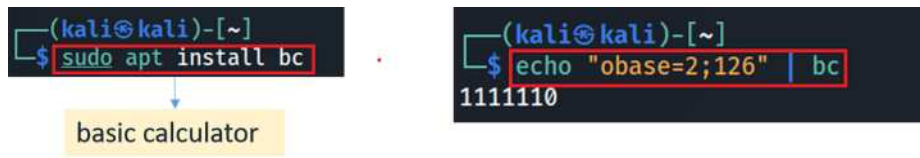


Figure 3.6: Linux command that converts decimal to binary

Using bc. This command is helpful when the investigator has a decimal value from a report or log and needs its binary form without doing the calculation manually. The pipe sends the text 'obase=2; 126' into `bc`, an arbitrary-precision calculator. Within that input, `obase=2` sets the output base to binary, and `126` is the decimal number being converted. The important output is '1111110', the binary equivalent of 126.

Decimal to hexadecimal. Figure 3.7 shows the manual repeated-division method for converting decimal numbers into hexadecimal values.

$$2738_{10} = (?)_{16}$$

16	2738	
16	171	2
16	10	11 = B
	0	10 = A

$$2738_{10} = AB2_{16}$$

Figure 3.7: Illustration of decimal to hexadecimal conversion

For the decimal value 2738, repeated division by 16 yields the hexadecimal digits A, B, and 2, producing ‘AB2’. Figure 3.8 shows two Linux methods for doing that same conversion directly.

```
(kali㉿kali)-[~]
$ echo "obase=16;2738" | bc
AB2
```

```
(kali㉿kali)-[~]
$ printf "%x\n" 2738
ab2
```

controls the output as in C printf

- %d - Print the argument as a signed decimal integer.
- %x - Print the argument as an unsigned hexadecimal integer.
- %s - Print the argument as a string.

Figure 3.8: Showcasing decimal to hexadecimal conversion

Using bc. The command `echo “obase=16; 2738” | bc` sends an instruction to `bc` to switch its output base to hexadecimal and then evaluate the decimal number 2738. The result to notice is ‘AB2’.

Using printf. The command `printf“%x\n”2738` formats the decimal number 2738 as hexadecimal and prints ‘ab2’. The `%x` specifier means “render this number in hexadecimal,” and `\n` adds a line break so the shell prompt stays readable. Investigators will often see lowercase hexadecimal in command output; the important point is the value, not the letter case.

Binary to hexadecimal. Investigators often move between binary and hexadecimal because hex dumps are more compact than long binary strings while still preserving exact byte values. Figure 3.9 shows the standard manual method: group the binary digits into sets of four, starting from the right, and convert each group to its hexadecimal equivalent.

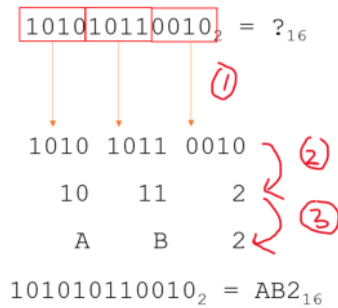


Figure 3.9: Illustration of binary to hexadecimal conversion

For example, the binary value ‘101010110010’ can be grouped as ‘1010 1011 0010’. These groups correspond to hexadecimal digits A, B, and 2, so the full hexadecimal value is ‘AB2’. This shortcut is especially useful when relating raw binary values in memory to the hexadecimal representations commonly displayed by forensic tools.

3.1.2 Python-Based Number System Conversions

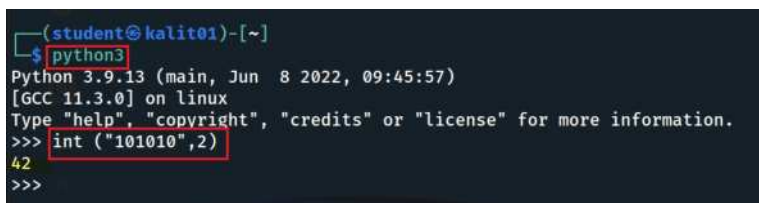
Python is useful when these conversions need to become part of a repeatable script rather than a one-off shell check. Before using the examples, Figure 3.10 confirms that Python 3 is available on the system.

```
(student@kalit01)-[~]  
$ python --version  
Python 2.7.18  
  
(student@kalit01)-[~]  
$ python3 --version  
Python 3.9.13
```

Figure 3.10: Verifying the installed Python versions

The version-check commands in Figure 3.10 report which Python interpreters are installed and which version number each one provides. The key thing to confirm is that Python 3 is available, because the examples in this chapter rely on Python 3 behavior and syntax.

Binary to decimal with int(). Figure 3.11 demonstrates `int("101010", 2)`, which tells Python to interpret the string '101010' as a base-2 value and convert it to an integer.

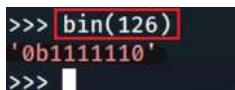
A terminal window with a dark background. The prompt is '(student@kalit01)-[~]'. The user enters 'python3'. The prompt changes to 'Python 3.9.13 (main, Jun 8 2022, 09:45:57) [GCC 11.3.0] on linux'. The user enters 'Type "help", "copyright", "credits" or "license" for more information.'. The user enters '>>> int("101010",2)'. The output is '42'. The user enters '>>>' again.

```
(student@kalit01)-[~]  
$ python3  
Python 3.9.13 (main, Jun 8 2022, 09:45:57)  
[GCC 11.3.0] on linux  
Type "help", "copyright", "credits" or "license" for more information.  
>>> int("101010",2)  
42  
>>>
```

Figure 3.11: Python command that converts binary (101010) to decimal (42)

The important parts are the binary string itself and the second argument, **2**, which specifies the base. The output '42' confirms the decimal interpretation.

Decimal to binary with bin(). Figure 3.12 shows the inverse operation with `bin(126)`.

A terminal window with a dark background. The user enters '>>> bin(126)'. The output is ''0b1111110''. The user enters '>>>' again.

```
>>> bin(126)  
'0b1111110'  
>>>
```

Figure 3.12: Convert decimal number to its binary equivalent in Python

Python returns '0b1111110'. The **0b** prefix marks the value as binary, and the remaining digits are the actual bit pattern. In other words, `bin()` returns a text-style representation of the number in base 2, which makes it convenient for quick checks, logging, or display inside larger forensic scripts.

3.2 ASCII: Bridging human-readable text and binary code

Beyond numerical conversions, investigators often need to determine whether raw bytes represent readable text, control characters, or encoded data. That makes character encoding a practical forensic skill rather than just a computing concept. **ASCII** (American Standard Code for Information Interchange) serves as

the bridge between human-readable characters and their underlying binary representations. It assigns unique numerical values to letters, numerals, punctuation marks, and control characters, enabling computers to store and process text.

The command `man ascii`, as illustrated in Figure 3.13, provides a comprehensive reference to the ASCII character set. An investigator might use this command when interpreting text recovered from a hex dump or when checking the numeric value associated with a control character.

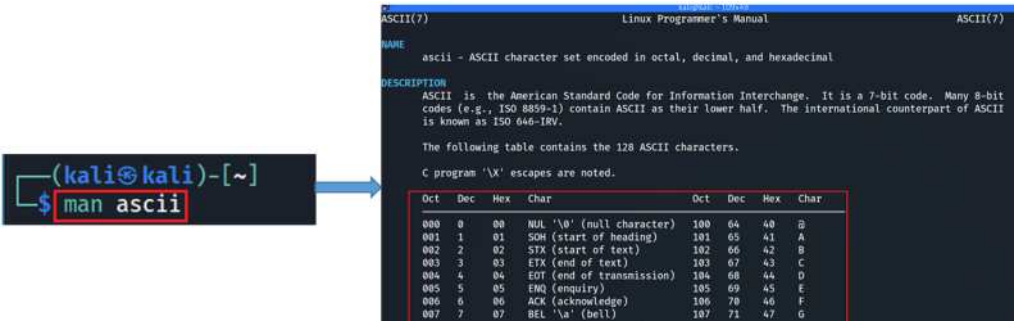


Figure 3.13: ASCII manual page in the terminal

The manual page output presents:

ASCII Table A tabular mapping between characters and their decimal, octal, and hexadecimal values.

Character Categories Explanations of control characters, printable characters, and other ASCII groupings.

In conclusion, proficiency in ASCII interpretation enables us to effectively analyze textual data, decipher hidden messages, and uncover crucial evidence embedded within the binary landscape of digital devices.

3.2.1 Understanding Character Encoding: ASCII vs. UTF-8

In the realm of digital forensics, understanding character encoding is vital, as it dictates how text data is represented and interpreted. **Unicode**, formally The “**Unicode Standard**” is an information technology standard for the consistent encoding, representation, and handling of text expressed in most of the world’s

writing systems.

Two common encoding standards are **ASCII** and **UTF-8**. While they both serve the purpose of representing characters using binary code, they differ significantly in their capabilities and applications.

ASCII. ASCII (American Standard Code for Information Interchange) remains important in digital forensics because many legacy artifacts, file headers, and control characters still rely on this limited character set.

- **Limited Character Set:** ASCII uses 7 bits to represent characters, allowing for a total of 128 unique codes. This is sufficient for basic English characters, numbers, and punctuation marks.
- **Simplicity:** Its simplicity and widespread adoption make ASCII a common choice for representing text in many legacy systems and file formats.
- **Inadequate for Non-English Text:** ASCII's limited character set makes it unsuitable for representing characters from other languages, leading to issues when dealing with multilingual data.

UTF-8. UTF-8 (Unicode Transformation Format - 8-bit) is more common in modern systems, so investigators need to recognize when text evidence extends beyond the ASCII range.

- **Extensive Character Set:** UTF-8 supports the entire Unicode character set, encompassing over 1 million characters from various languages and scripts worldwide.
- **Variable-Length Encoding:** UTF-8 employs a clever variable-length encoding scheme, using 1 to 4 bytes to represent different characters. This allows for efficient storage of both ASCII and non-ASCII characters.
- **Backward Compatibility:** UTF-8 is designed to be backward compatible with ASCII. The first 128 characters in UTF-8 have the same representation as in ASCII, ensuring seamless handling of basic English text.
- **Universal Adoption:** UTF-8's ability to handle a vast array of characters has led to its widespread adoption as the de facto standard for text encoding on the web and in modern computing systems.

With that context, we can move from character tables to practical conversions. In forensic work, these conversions help transform raw numeric values into readable text and, just as importantly, show when apparently readable text is actually

stored as hexadecimal bytes.

3.2.2 Linux ASCII Conversions

When an investigator sees suspicious bytes or recovered text fragments, Linux provides quick ways to translate between numeric values and readable characters. These conversions are especially helpful when validating what a hex dump or carved string actually represents.

Hexadecimal to ASCII with printf. Figure 3.14 uses the `printf` command with the hexadecimal byte sequence `\x48\x65\x6c\x6c\x66` to decode byte values into text.



Figure 3.14: Linux command that converts hexadecimal to ASCII

Each `\x` escape introduces one byte written as two hexadecimal digits, so `\x48` becomes ‘H’, `\x65` becomes ‘e’, and so on. The important output is ‘Hello’, which confirms that the recovered bytes decode to meaningful text.

Decimal to ASCII with nested printf. Figure 3.15 demonstrates `printf "\x$(printf "%x" "72")"`, which converts the decimal value 72 into the character ‘H’.

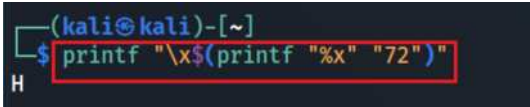
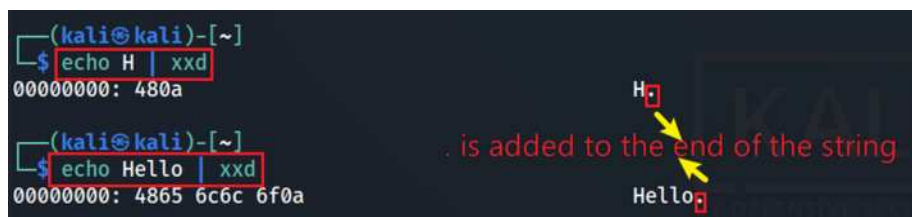


Figure 3.15: Decimal-to-ASCII conversion using nested printf command

The inner `printf "%x" "72"` converts decimal 72 into hexadecimal ‘48’. The `$(...)` syntax is command substitution: the shell runs the inner command first and inserts its output into the outer command. That means the shell effectively

builds `\x48`, and the outer `printf` then interprets that byte value and prints the corresponding character. This pattern is useful when evidence stores character values in decimal form but the shell expects byte-style hexadecimal escapes.

ASCII to hexadecimal with `xxd`. Figure 3.16 shows the reverse operation using `echo 'H' | xxd`.



```
(kali㉿kali)-[~]  
$ echo H | xxd  
00000000: 480a  
  
(kali㉿kali)-[~]  
$ echo Hello | xxd  
00000000: 4865 6c6c 6f0a
```

Figure 3.16: Linux command that converts ASCII into hexadecimal

Here, `xxd` creates a hexadecimal dump of the incoming bytes. The output `'00000000: 480a'` shows that `'H'` is stored as `48`, while `0a` is the newline added by `echo`. If the investigator wants only the character bytes, `echo -n` avoids the trailing newline. The same approach works for longer text such as `'Hello'`, producing the byte sequence `48 65 6c 6c 6f`.

3.3 ASCII Conversions in Python 3

Continuing our exploration of character encoding transformations within Python, we now shift to Python 3 and its support for working directly with bytes. This matters when an investigator needs to decode recovered values, compare byte sequences, or feed converted text directly into the next parsing step of a script. In forensic scripts, this is often cleaner than shell pipelines because the decoded or encoded values can be reused immediately in later parsing steps.

The `binascii` module provides a straightforward bridge between raw bytes and hexadecimal text.

ASCII to hexadecimal with `hexlify()`. Figure 3.17 illustrates this conversion. The example call `binascii.hexlify(b'H')` converts a bytes object into its hexadecimal representation.

The `b` prefix in `b'H'` matters because it creates a bytes object rather than a

```
>>> import binascii
>>> binascii.hexlify(b'H')
b'48' ←
>>> binascii.hexlify(b'Hello')
b'48656c6c6f' ←
```

Figure 3.17: Demonstrating ASCII to hexadecimal conversion in Python 3

Unicode string. `hexlify()` then returns the hexadecimal byte sequence, here shown as `b'48'`. The same pattern converts longer text such as `b'Hello'` into `b'48656c6c6f'`.

Hexadecimal to ASCII with `unhexlify()`. Figure 3.18 shows the reverse operation using `binascii.unhexlify('48')`.

```
>>> binascii.unhexlify('48')
b'H' ←
>>> binascii.unhexlify('48656c6c6f')
b'Hello' ←
```

Figure 3.18: Demonstrating hexadecimal to ASCII conversion in Python 3

`unhexlify()` takes the hexadecimal string, decodes each pair of digits into a byte, and returns the result as a bytes object. In this example, the output is `b'H'`. When the input is `'48656c6c6f'`, the output becomes `b'Hello'`. These byte-oriented results are especially useful when the next step in the script is parsing or comparing raw artifact data.

3.4 Understanding UNIX time: A numerical representation of time

Transitioning from character encoding, we now introduce another crucial concept in digital forensics: **UNIX time**. This system provides a standardized way to represent points in time, which is essential for interpreting and correlating timestamps across different systems and applications.

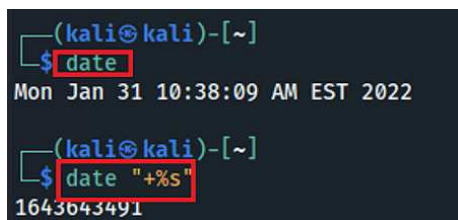
UNIX time, also known as “Epoch time” or “POSIX time”, represents the num-

ber of seconds that have elapsed since **January 1st, 1970 at 00:00:00** Coordinated Universal Time (UTC). This seemingly simple numerical representation serves as the backbone for timestamping in most modern operating systems and file systems.

In this section, we will delve into the practical aspects of working with UNIX timestamps in the Linux environment. We'll explore commands and techniques to convert between UNIX timestamps and human-readable date/time formats, empowering you to extract meaningful temporal information from digital evidence.

The Linux **date** command is often the quickest way to move between a readable time and its stored UNIX timestamp. This is useful when building a timeline from shell history, log entries, or file metadata.

Getting the current UNIX timestamp. Figure 3.19 shows **date** “**+%s**”, which prints the current time as the number of seconds since January 1, 1970 at 00:00:00 UTC.

A terminal window with a dark background. The prompt is (kali@kali)-[~]. The first command is \$ date, which is highlighted with a red box. The output is Mon Jan 31 10:38:09 AM EST 2022. The second command is \$ date +%s, which is also highlighted with a red box. The output is 1643643491.

```
(kali@kali)-[~]  
$ date  
Mon Jan 31 10:38:09 AM EST 2022  
  
(kali@kali)-[~]  
$ date +%s  
1643643491
```

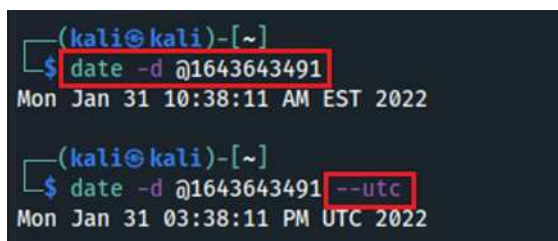
Figure 3.19: Date command to obtain the current UNIX timestamp

The key syntax is **+%s**. It tells **date** to format its output as epoch seconds rather than as a human-readable string. Before relying on this output in a forensic workflow, confirm that the system clock is accurate. If higher precision is needed, **%s.%N** adds nanoseconds.

Converting a UNIX timestamp to readable time. Figure 3.20 shows two versions of the conversion command.

Using local time. The command **date -d @1643643491** tells **date** to interpret the value after **@** as a UNIX timestamp in seconds. The output is shown in the system's local timezone.

Using UTC. The command **date -u -d @1643643491** converts the same times-



```
(kali@kali)-[~]  
$ date -d @1643643491  
Mon Jan 31 10:38:11 AM EST 2022  
  
(kali@kali)-[~]  
$ date -d @1643643491 --utc  
Mon Jan 31 03:38:11 PM UTC 2022
```

Figure 3.20: Converting UNIX timestamps to human-readable date and time format

tamp but forces the display into Coordinated Universal Time. The important lesson is that the numeric timestamp does not change; only the timezone used to display it changes. That distinction matters whenever the investigator is correlating artifacts from multiple systems.

3.5 UNIX Time (Python version)

Building upon our exploration of UNIX time in Linux, let's transition to Python, a versatile programming language frequently leveraged in digital forensics. This shift is useful when timestamp interpretation needs to become part of a repeatable workflow rather than a one-time terminal check. Python provides built-in modules and functions for handling UNIX timestamps, allowing for seamless integration of timestamp conversions and manipulations into investigative workflows.

In this section, we will delve into Python's capabilities for working with UNIX time, including converting between timestamps and human-readable formats, extracting specific date and time components, and performing calculations involving timestamps. By harnessing the power and flexibility of Python, forensic analysts can efficiently process and analyze the temporal aspects of digital evidence, gaining valuable insights into the timeline of events.

3.5.1 Getting Current Time and Timestamps

Python is a good fit when timestamp work needs to scale beyond a single shell command. The examples in this subsection show how to capture the current time

in both readable and numeric forms so it can be logged, compared, or reused in a script.

Using `datetime.now().timestamp()`. Figure 3.21 demonstrates `datetime.now().timestamp()`, which returns the current local time as a UNIX timestamp.

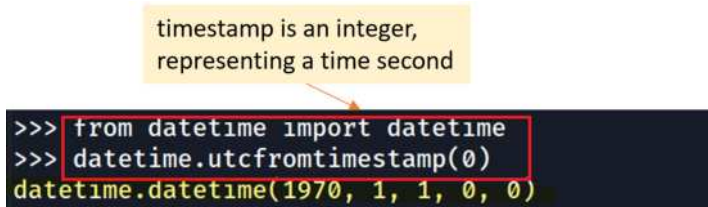


Figure 3.21: Obtaining the current UNIX timestamp

The import `from datetime import datetime` provides access to the `datetime` class. The call `datetime.now()` gets the current local date and time. The chained method `.timestamp()` then converts that value into seconds since the UNIX epoch. The result is usually a floating-point value, so fractional seconds are preserved.

Comparing local time and UTC. Figure 3.22 extends that idea by showing `datetime.now()` alongside `datetime.now(timezone.utc)`.

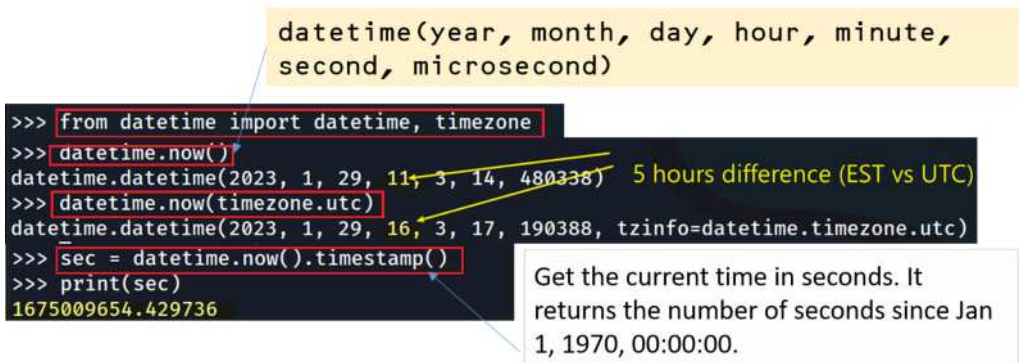


Figure 3.22: Demonstrating the use of the `datetime` module to obtain the current time in local and UTC timezones, as well as convert it to a UNIX timestamp

The important lesson is that Python can represent the same moment either as local time or as a timezone-aware UTC value. That distinction matters when evidence from multiple systems must be normalized before comparison.

Using the time module. Figure 3.23 shows the lighter-weight **time** module alternative.

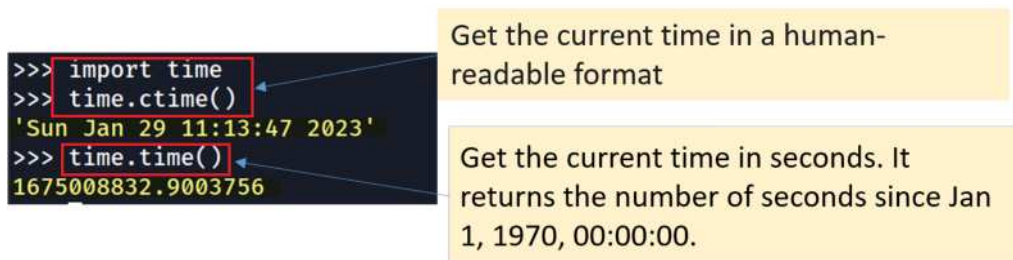


Figure 3.23: Python code using the time module to obtain the current time in both human-readable format and as a UNIX timestamp

`time.ctime()` returns the current local time as a readable string, while `time.time()` returns epoch seconds. This is convenient when a script needs only a quick timestamp and not a full **datetime** object.

3.5.2 Converting Timestamps to Readable Time

Forensic artifacts often store time as a raw number, so the next step is usually to make that value readable without losing track of timezone assumptions.

Using datetime.fromtimestamp(). Figure 3.24 shows this conversion step. The method `datetime.fromtimestamp()` converts a UNIX timestamp into a local **datetime** object.

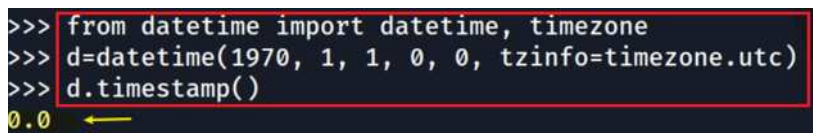


Figure 3.24: Converting a UNIX timestamp to a human-readable datetime object

This is the most direct way to turn a recovered UNIX value into something an investigator can place on a timeline. The key output is the readable date and time, but it should be interpreted as local time unless the code explicitly specifies otherwise.

Converting recovered timestamps. Figure 3.25 applies the same conversion pattern to a specific evidence-style value. In this example, the timestamp is 1675008832.

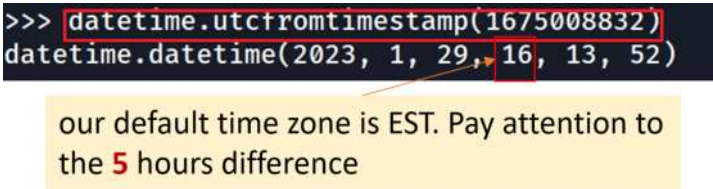


Figure 3.25: Converting a UNIX timestamp to a human-readable datetime object

The output `datetime.datetime(2023,1,29,11,13,52)` is the local-time interpretation of that timestamp. This is a reminder that the stored UNIX value is timezone-neutral, while the displayed result depends on the conversion context.

Using `datetime.datetime.fromtimestamp()`. Figure 3.26 shows the UTC-specific conversion with `datetime.datetime.fromtimestamp(1675008832)`.

If the timestamps are in UTC timezone (a common way to store dates) you should use

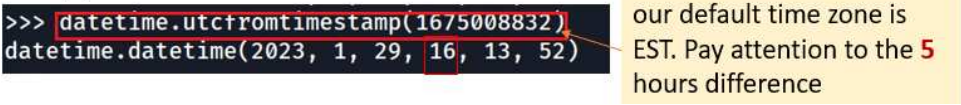


Figure 3.26: Demonstrating the conversion of a UNIX timestamp to a UTC date-time object

This form is useful when building a normalized cross-system timeline, because it returns the timestamp as a UTC datetime rather than as local time. For evidentiary interpretation, that explicitness is often safer than relying on the host system’s timezone settings.

3.5.3 Comparing Epochs and Time Differences

Some investigations require more than display conversion. They require comparing two timestamp systems and calculating the offset between them.

How the Script Works. Figure 3.27 explains a small Python script, `time_diff.py`, that calculates the offset between the UNIX epoch (January 1, 1970) and the iOS Cocoa Core Data epoch (January 1, 2001).

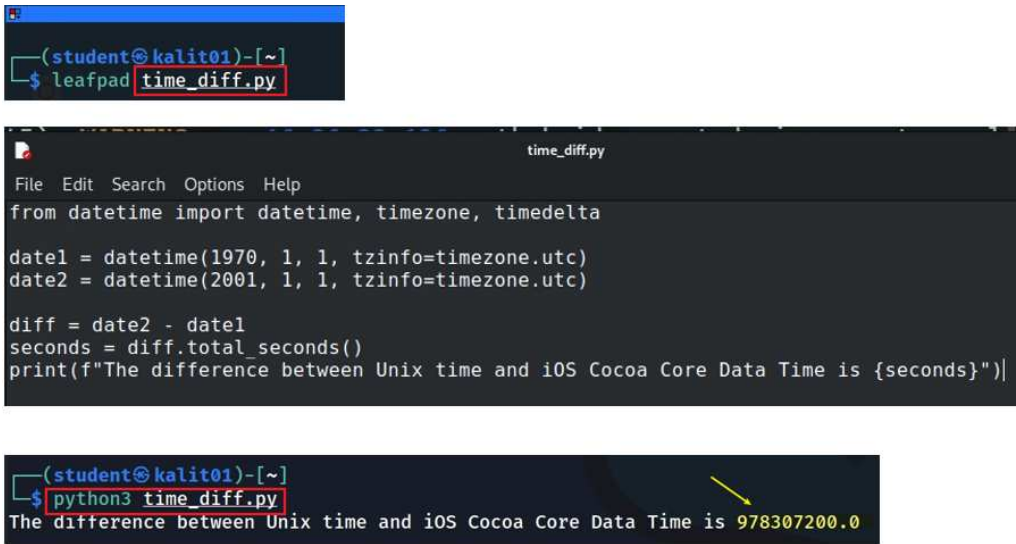


Figure 3.27: Calculating the time difference between UNIX epoch and iOS Cocoa Core Data time reference

The script imports `datetime`, `timezone`, and `timedelta`, creates the two reference dates in UTC, subtracts them, and then calls `total_seconds()` on the resulting `timedelta`. The important output is **978,307,200 seconds**, which is the offset an investigator must account for when translating between these two timestamp systems.

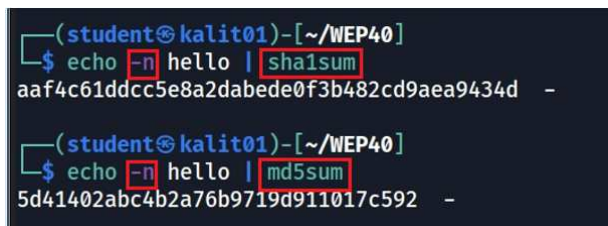
3.6 Understanding hash codes in investigations

Moving beyond character encoding and timestamp analysis, another critical aspect of digital forensics involves verifying the integrity and authenticity of data. Investigators rely on hashes when documenting evidence, checking whether files changed, and comparing recovered data against known values. **Hash functions** play a pivotal role in this regard, providing a mechanism to generate unique digital fingerprints for files or data, irrespective of their size or complexity.

A **hash function** takes an input (data or file) and produces a fixed-size output, known as a hash value or digest. This hash value acts as a unique identifier for the input data. Even the slightest change in the input will result in a drastically different hash value, making hash functions invaluable for ensuring data integrity and facilitating comparisons.

3.6.1 Command-Line Hashing

Hash functions, as discussed previously, provide a mechanism to generate unique digital fingerprints for files or data. The terminal commands demonstrated in Figure 3.28 showcase the use of two widely-used hashing algorithms, “SHA-1” and “MD5”, to compute hash codes for the string ‘hello’ in a Linux environment.

A terminal window with a dark background. The prompt is `(student@kalit01)-[~/WEP40]`. The first command is `$ echo -n hello | sha1sum`, and the output is `aaf4c61ddcc5e8a2dabede0f3b482cd9aea9434d -`. The second command is `$ echo -n hello | md5sum`, and the output is `5d41402abc4b2a76b9719d911017c592 -`.

```
(student@kalit01)-[~/WEP40]
$ echo -n hello | sha1sum
aaf4c61ddcc5e8a2dabede0f3b482cd9aea9434d -

(student@kalit01)-[~/WEP40]
$ echo -n hello | md5sum
5d41402abc4b2a76b9719d911017c592 -
```

Figure 3.28: Calculating SHA-1 and MD5 hashes for the string “hello”

Using SHA-1. The command `echo -n hello | sha1sum` prints the string ‘hello’ without a trailing newline and pipes those bytes into `sha1sum`. The important output is the digest `aaf4c61ddcc5e8a2dabede0f3b482cd9aea9434d`. The

trailing hyphen indicates that the bytes came from standard input rather than from a named file.

Using MD5. The command `echo -n hello | md5sum` follows the same pattern but uses `md5sum` instead. The important output is `5d41402abc4b2a76b9719d911017c592`. In both examples, the pipe passes the evidence bytes directly to the hash program, and the digest is the part an investigator would compare or document.

The ability to generate and compare hash codes is a fundamental skill for digital forensic analysts. The `sha1sum` and `md5sum` commands, as showcased in Figure 3.28, provide readily available tools for calculating hashes in a Linux environment. While these specific algorithms might have limitations in terms of cryptographic strength, they remain relevant in various forensic scenarios for verifying data integrity and facilitating evidence identification.

3.6.2 Python Hashing

In addition to command-line tools, Python provides robust capabilities for calculating hash codes, a critical aspect of digital forensics for verifying data integrity and identifying files. Figure 3.29 showcases the use of Python’s **hashlib** module to generate “SHA-1” and “MD5” hashes for the string “hello”.

Using MD5 in Python. The call

```
hashlib.md5("hello".encode('utf-8')).hexdigest()
```

follows the same pattern and returns `5d41...c592`. The key idea is that once the data is in bytes, the same workflow can be reused for stronger algorithms such as SHA-256.

While **MD5** and **SHA-1** are illustrated here, it’s important to remember that **SHA-256** or stronger algorithms are generally recommended for modern forensic applications due to the increasing vulnerabilities of older hash functions.

```
1 import hashlib
2 hashlib.sha1("hello".encode('utf-8')).hexdigest()
3
✓ 0.0s

'aaf4c61ddcc5e8a2dabede0f3b482cd9aea9434d'

1 hashlib.md5("hello".encode('utf-8')).hexdigest()
✓ 0.0s

'5d41402abc4b2a76b9719d911017c592'
```

Figure 3.29: Calculating SHA-1 and MD5 hashes for a string “hello”

3.7 The significance of byte order in digital forensics

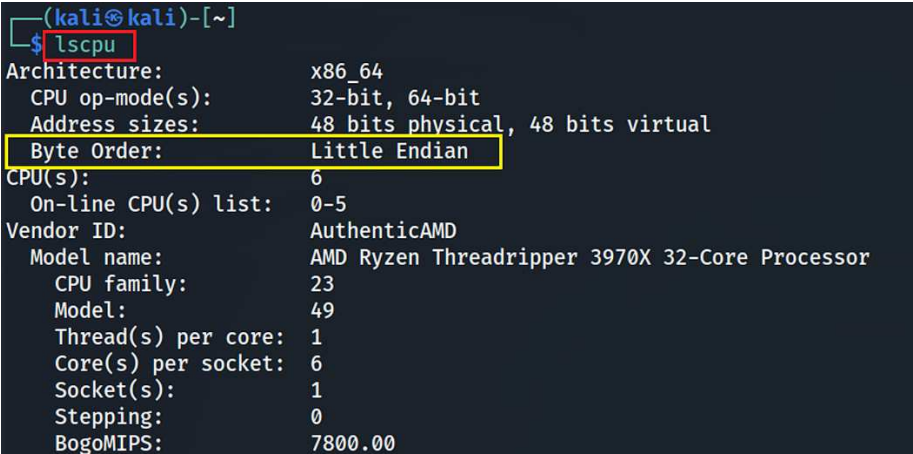
In digital forensics, a crucial aspect of data analysis lies in understanding how data is organized and stored in memory, particularly when dealing with multi-byte values. Misreading byte order can completely change the meaning of a timestamp, file signature, length field, or integer recovered from raw evidence. This arrangement, known as byte order or endianness, directly impacts how data is interpreted and can significantly influence the accuracy of forensic investigations.

While the previous sections have focused on the representation and conversion of numerical data, this section highlights a critical aspect of how data is stored in memory: byte order, also known as endianness.

3.7.1 Byte order identification

Figure 3.30 presents the output of the `lscpu` command. An investigator might run `lscpu` to confirm how the current system stores multi-byte values before interpreting raw bytes from memory, disk structures, or carved data. The command reports several CPU characteristics, but for this discussion the key line to

notice is **Byte Order**. If the output says **Little Endian**, the least significant byte is stored first in memory; if it says **Big Endian**, the most significant byte is stored first. That single line gives the reader the information needed to interpret multi-byte values correctly in later forensic analysis.



```
(kali@kali)-[~]  
$ lscpu  
Architecture:          x86_64  
CPU op-mode(s):        32-bit, 64-bit  
Address sizes:         48 bits physical, 48 bits virtual  
Byte Order:            Little Endian  
CPU(s):                6  
On-line CPU(s) list:   0-5  
Vendor ID:             AuthenticAMD  
Model name:            AMD Ryzen Threadripper 3970X 32-Core Processor  
CPU family:            23  
Model:                 49  
Thread(s) per core:    1  
Core(s) per socket:    6  
Socket(s):             1  
Stepping:              0  
BogoMIPS:              7800.00
```

Figure 3.30: Output of the lscpu command showing the system’s byte order

3.7.2 Contrasting byte ordering schemes: Little Endian vs. Big Endian

As demonstrated in the previous output (Figure 3.30), the line “Byte Order: Little Endian” reveals that the system under investigation utilizes the little-endian format.

The concept of endianness as demonstrated in Figure 3.31 revolves around how the bytes within a multi-byte data type (such as integers or floating-point numbers) are arranged in memory.

- **Little Endian:** In a little-endian system, the least significant byte (LSB), which holds the smallest positional value within the multi-byte data, is stored at the lowest memory address. As we see in Figure 3.31, the byte 0x04 (the LSB) occupies the address 0x100, while the most significant byte (MSB), 0x01, is stored at the highest address, 0x103. This arrangement might seem counter-intuitive to humans who typically read and write numbers from left to right (most significant



Figure 3.31: Illustration of little endian and big endian byte ordering

digit first).

- **Big Endian:** In contrast, big-endian systems prioritize the most significant byte (MSB), placing it at the lowest memory address. Figure 3.31 illustrates this by showing 0x01 (the MSB) at address 0x100 and 0x04 (the LSB) at 0x103. This ordering aligns more closely with our conventional understanding of numerical representation.

3.8 Summary

This chapter introduced the core data representations that investigators encounter when moving between raw evidence and human-readable interpretation. We examined how to convert values among binary, decimal, and hexadecimal; how to translate between bytes and text using ASCII and UTF-8 concepts; how to interpret UNIX timestamps and compare time systems; how to use hashes to document and verify integrity; and how byte order affects the meaning of multi-byte values.

Together, these skills support everyday forensic tasks such as reading hex dumps, validating encoded text, normalizing timestamps across systems, comparing evidence with known values, and correctly interpreting low-level structures. The Linux and Python examples in this chapter are meant to provide quick reference points that analysts can reuse when explaining findings, checking assumptions, and building repeatable workflows.

Chapter Review

Review Questions

1. Why is hexadecimal commonly used when examining raw digital evidence?
2. Convert the hexadecimal value 2A to decimal and binary.
3. What information does a UNIX timestamp represent, and why should its timezone be recorded?
4. What can matching hash values support, and what do they not prove?
5. How can incorrect byte-order interpretation change a forensic conclusion?

Discussion Questions

1. What safeguards should an analyst use before relying on a converted timestamp in a report?
2. Why should hash comparisons be combined with documentation rather than treated as a complete chain of custody?

Hands-On Exercise

Exercise. Using the Linux or Python examples already introduced in this chapter, convert 10110110, 7F, and decimal 255 into the other relevant number systems. Convert one UNIX timestamp to UTC and local time, calculate a hash for a practice file, and record the commands and results in an analyst note.

4 Computer Systems and Digital Evidence

Learning Objectives

- Describe the hardware, firmware, operating-system, application, and user layers of a computer system.
- Identify potential evidence sources associated with processors, memory, storage, network interfaces, and peripherals.
- Distinguish sectors, partitions, Master Boot Records, and GUID Partition Tables at a conceptual level.
- Explain the legacy BIOS/MBR and modern UEFI/GPT boot paths.
- Identify the boot and partition-table observations that should be documented during an examination.

Digital evidence is created by a system of connected layers, not by one application alone. A user's action can leave traces in an application, the operating system, memory, storage, network records, and attached devices. Before an examiner uses a forensic tool, it is helpful to understand where those traces originate and what each layer can and cannot show. This chapter provides that orientation for the later hands-on work in the book.

4.1 Computer systems as evidence environments

A computer system combines physical components with software that coordinates them. The hardware includes processors, memory, storage, network interfaces,

and peripheral devices. Firmware initializes hardware; the operating system manages resources; applications perform user-facing work; and users or automated processes create activity. Each layer can create useful artifacts, and the relationship among the layers helps an investigator test an explanation rather than rely on a single trace [Kent et al., 2006].

Figure 4.1 shows the four computer-system layers used throughout this chapter: user, applications, operating system, and device or computer hardware. A single user action can create related artifacts at more than one layer, so an examiner should use the figure as a guide to possible evidence sources rather than as a guarantee that a particular trace exists.

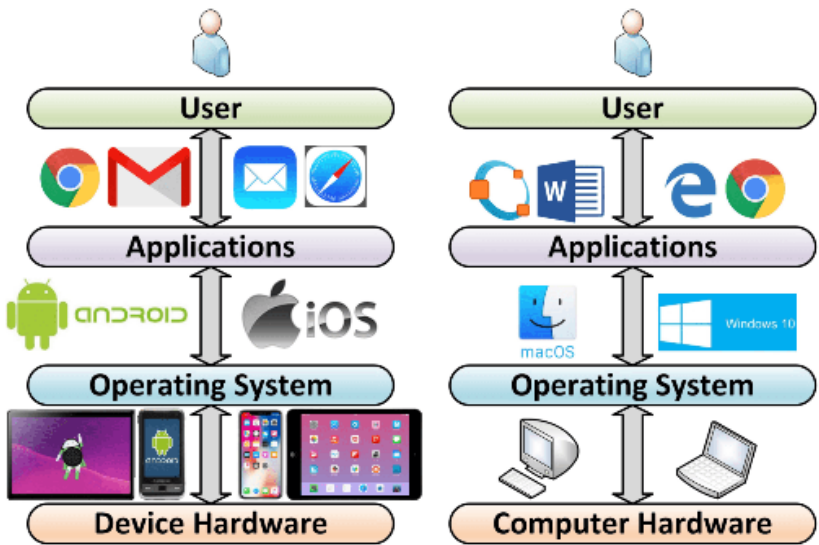


Figure 4.1: Computer-system layers. User activity can generate related evidence in applications, operating systems, and hardware.

Table 4.1: Computer-system layers and examples of the evidence each layer may generate.

Layer	Role	Examples of potential evidence
-------	------	--------------------------------

User	Initiates, directs, or automates activity through an account or interface.	Account activity, authentication records, input history, and actions attributed to a user context.
Applications	Create, view, transmit, and modify information for a user or service.	Documents, browser history, application logs, account activity, and recent-file lists.
Operating system	Schedules work and manages files, memory, devices, and network access.	Event logs, process information, user accounts, configuration data, and file metadata.
Device or computer hardware	Processes, retains, and communicates data through components such as memory, storage, network interfaces, and peripherals.	Memory contents, disk sectors, partitions, device serial numbers, network-interface information, and removable-media traces.

Table 4.1 uses the same four-layer model as the figure. It is a reasoning aid, not a guarantee that every artifact exists or is reliable. A browser record may indicate that a URL was stored in a profile, while network records may show that data was transmitted. Neither observation alone proves who was at the keyboard or why an action occurred. Stronger conclusions come from correlating artifacts across layers and documenting their limitations.

4.2 Core components and their forensic relevance

Before examining individual artifacts, students should be able to locate the major components of a computer system and explain their basic roles. Figure 4.2, adapted from the Carnegie Cyber Academy’s *How Computers Work* resource, identifies the CPU, memory, storage, network interface, motherboard, and con-

nected peripherals [Carnegie Cyber Academy, 2026]. In a forensic examination, these components are potential sources of distinct artifacts, not proof of a particular user action.

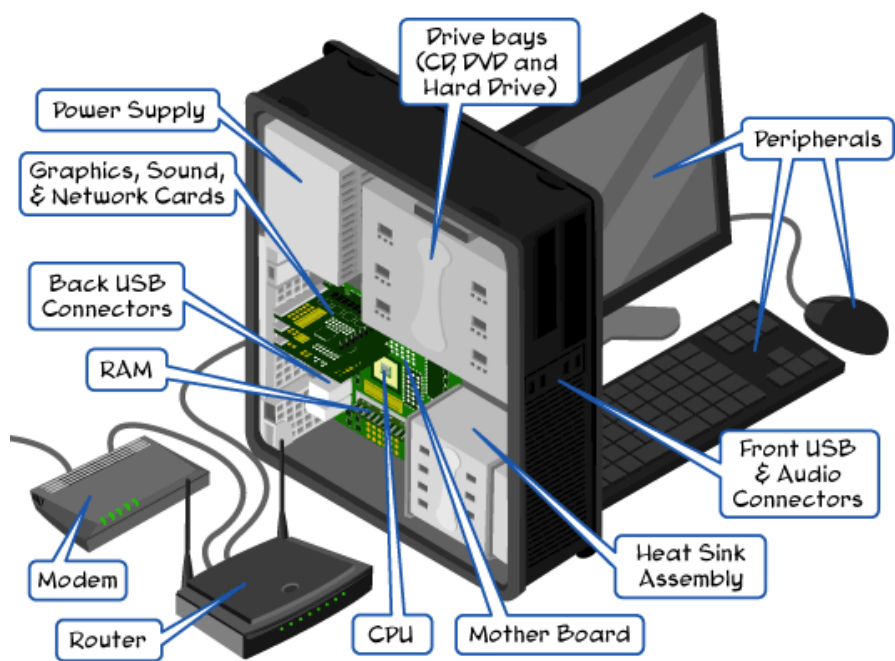
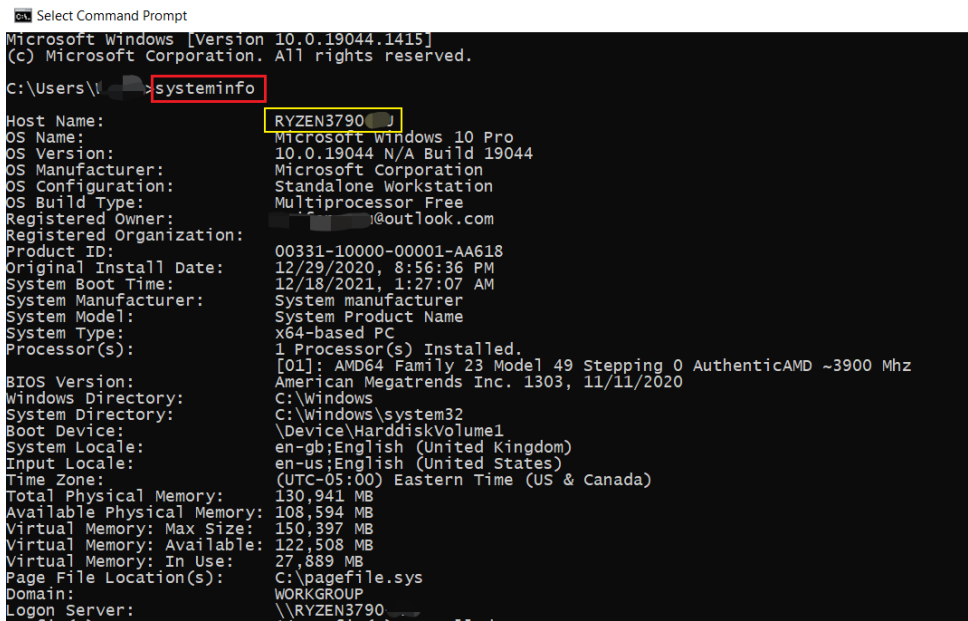


Figure 4.2: High-level computer components, including the CPU, RAM, motherboard, storage drive bays, network card, and peripherals. Reproduced with permission from Carnegie Cyber Academy, *How Computers Work* [Carnegie Cyber Academy, 2026].

Processor and memory. The central processing unit executes instructions, and memory holds data needed by running processes. Memory can contain volatile evidence such as active processes, network connections, decrypted material, and unsaved content. Because this information can disappear when power is removed, collection decisions for a running system require authorization, documented judgment, and attention to the risk of altering evidence.

Figure 4.3 shows a command-line system-information report. Its value is not the particular values on the screen, but the categories an examiner should record: operating-system and kernel information, processor architecture, memory, stor-

age, and network interfaces.



```
Microsoft Windows [Version 10.0.19044.1415]
(c) Microsoft Corporation. All rights reserved.

C:\Users\...\>systeminfo

Host Name:                               RYZEN3790-J
OS Name:                                 Microsoft Windows 10 Pro
OS Version:                             10.0.19044 N/A Build 19044
OS Manufacturer:                       Microsoft Corporation
OS Configuration:                     Standalone Workstation
OS Build Type:                          Multiprocessor Free
Registered Owner:                       [REDACTED]@outlook.com
Registered Organization:                [REDACTED]
Product ID:                             00331-10000-00001-AA618
Original Install Date:                  12/29/2020, 8:56:36 PM
System Boot Time:                       12/18/2021, 1:27:07 AM
System Manufacturer:                   System manufacturer
System Model:                           System Product Name
System Type:                            x64-based PC
Processor(s):                           1 Processor(s) Installed.
[01]: AMD64 Family 23 Model 49 Stepping 0 AuthenticAMD ~3900 Mhz
BIOS Version:                           American Megatrends Inc. 1303, 11/11/2020
Windows Directory:                     C:\Windows
System Directory:                       C:\Windows\system32
Boot Device:                            \Device\HarddiskVolume1
System Locale:                           en-gb:English (United Kingdom)
Input Locale:                           en-us:English (United States)
Time Zone:                              (UTC-05:00) Eastern Time (US & Canada)
Total Physical Memory:                  130,941 MB
Available Physical Memory:              108,594 MB
Virtual Memory: Max Size:               150,397 MB
Virtual Memory: Available:              122,508 MB
Virtual Memory: In Use:                 27,889 MB
Page File Location(s):                  C:\pagefile.sys
Domain:                                 WORKGROUP
Logon Server:                           \\RYZEN3790-...
```

Figure 4.3: A command-line system-information report. Record the observed configuration and time of collection; do not infer user activity from configuration alone.

Storage. Hard drives, solid-state drives, USB devices, and memory cards retain data after power is removed. Their contents can include allocated files, deleted content, unallocated space, metadata, and system configuration. A forensic image captures storage at a point in time so that later examination can proceed from a working copy rather than the original media.

Network interfaces and peripherals. Network adapters, printers, cameras, phones, external drives, and other peripherals can expand the evidence environment. They may introduce identifiers, connection records, transferred content, or storage outside the primary computer. An examiner should record which devices are present and avoid assuming that an artifact on one device explains all activity across the system.

4.3 Runtime system context and evidence correlation

The components described above work together while a system is running. Figure 4.4 distinguishes a stored program from a process, its executing instance, and the threads through which that process performs work. The operating system schedules those threads on the processor, allocates memory, records relevant events, and coordinates access to storage, network interfaces, and peripheral devices.

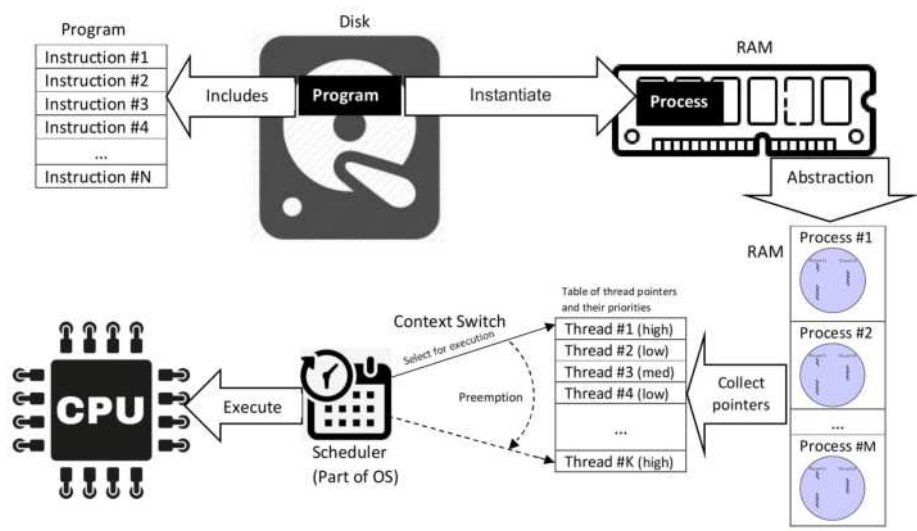


Figure 4.4: Program, process, and thread relationships. The relationship helps explain why execution, scheduling, and memory state can be valuable volatile evidence.

The process-monitoring view in Figure 4.5 provides a complementary runtime snapshot. Process names, process identifiers, parent-child relationships, and resource use can help guide an investigation, but the display is time-sensitive and requires corroboration from other artifacts.

The runtime model maps directly to several common forensic tasks:

Memory forensics. A RAM capture can preserve volatile processes, network

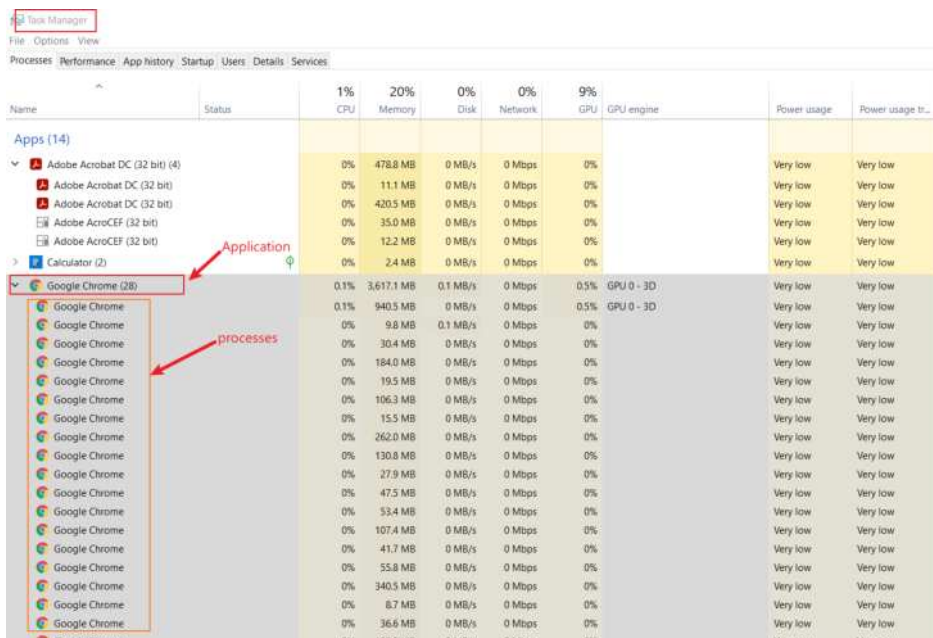


Figure 4.5: A process-analysis interface. Treat a process list as a snapshot of a running system and document the tool, time, and collection conditions.

connections, decrypted material, unsaved content, and sometimes injected code.

Process analysis. Process identifiers, parent-child relationships, command lines, loaded modules, and resource use can help reconstruct program execution.

Operating-system log analysis. Event records, authentication activity, service changes, and device connections provide system-level context.

Application-log and artifact analysis. Browser, email, office, and other application records may show user-facing activity and are often persisted to storage.

Static analysis. Inspect an executable without running it. Its metadata, hashes, strings, imports, and code structure can indicate potentially malicious capability.

Static program analysis is not a runtime task, but it complements process and memory analysis by connecting an observed executable to its possible behavior. Each task has limits and should be interpreted alongside other artifacts.

Consider a user who opens a browser and downloads a document. The application may create one or more processes and threads; the processor executes them and memory can hold transient content, connection details, or decrypted data. The operating system may record process, account, and event information; storage may retain the downloaded file, cache data, and metadata; and network records may show the connection or transfer. Each source answers a different part of the evidence question, so an examiner should correlate them rather than treat any one artifact as conclusive.

Artifact availability varies by system. Operating-system and application versions, cloud synchronization, encryption, hardware, and connected devices all affect what is retained. A missing artifact may reflect configuration or normal system behavior rather than deliberate concealment. Document the device type, operating-system version, relevant accounts, attached media, network state, collection time and timezone, and tools used. This record helps another examiner understand which conclusions are supported and which questions remain open.

4.4 Hard disks as forensic evidence sources

Hard disk drives (HDDs) store data on rotating magnetic platters. Figure 4.6 shows the internal components of a typical HDD: platters retain data, a spindle rotates them, and an actuator positions read/write heads over their surfaces. The electronics board and firmware coordinate those mechanical components with the host computer.



Figure 4.6: Internal components of a hard disk drive. Platters, the spindle, the actuator, and read/write heads provide physical context for sector-based evidence [Reg Media, 2006].

Figure 4.7 relates the visible drive components to the storage units used during examination. The platter is the magnetic surface that stores data; a track is a circular path on that surface; and a sector is an addressable portion of a track. The actuator moves the read/write heads to the required location. This geometry is a conceptual model: modern drives normally present logical block addresses (LBAs) rather than exposing a physical track-and-sector location. Nevertheless, sector numbers and byte offsets let an examiner describe where data was observed even when a filename is unavailable.

Physical and logical storage. A sector is the smallest addressable unit that

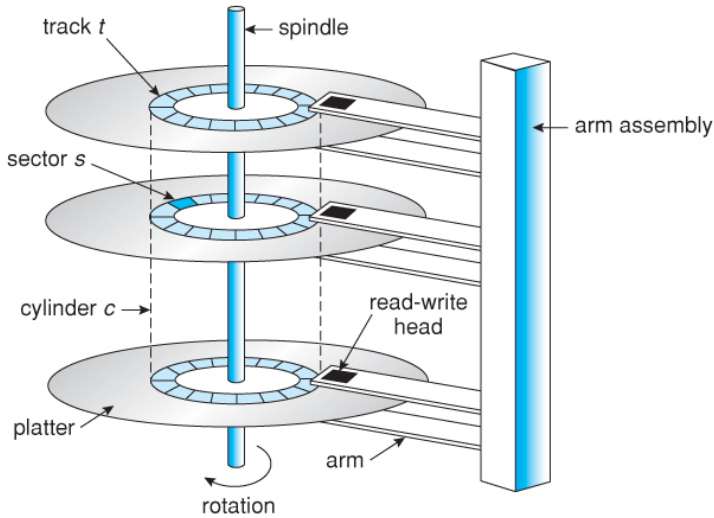


Figure 4.7: Hard-disk geometry. The labeled platter, track, sector, and read/write head provide context for sector- and offset-based examination [Silberschatz et al., 2013].

a drive normally exposes to the operating system. Many drives use 512-byte logical sectors, while others use 4,096-byte sectors or a 512-byte logical interface over 4,096-byte physical sectors. Operating systems present these sectors through logical structures, including partitions and volumes, which the next section introduces.

Capacity calculation. For a device with 81,600,000 sectors and 512 bytes per logical sector, multiply the sector count by the sector size:

$$81,600,000 \text{ sectors} \times 512 \text{ bytes/sector} = 41,779,200,000 \text{ bytes.}$$

Using binary multiples, 1 KiB = 1,024 bytes, 1 MiB = 1,024 KiB, and 1 GiB = 1,024 MiB. The capacity is therefore:

$$\begin{aligned} & 41,779,200,000 \text{ bytes} \div 1,024 \\ &= 40,800,000 \text{ KiB,} \quad 40,800,000 \text{ KiB} \div 1,024 = 39,843.75 \text{ MiB,} \\ & \quad 39,843.75 \text{ MiB} \div 1,024 \approx 38.91 \text{ GiB.} \end{aligned}$$

Drive manufacturers commonly use decimal units, where 1 GB = 1,000,000,000 bytes. The same value is therefore approximately 41.78 GB in decimal notation. Record both the sector count and the sector size in an evidence note; a displayed capacity alone does not identify the source device.

Figure 4.8 is a disk surface-scan interface. Its grid summarizes tested regions of the drive: green blocks indicate that the scan did not report a read error for that region, while red blocks would indicate a reported problem. The display does not show raw sector bytes, does not identify the contents of a region, and does not by itself verify a forensic image. It can, however, help an examiner document observed media condition alongside the device identity, scan settings, time, and any acquisition results.

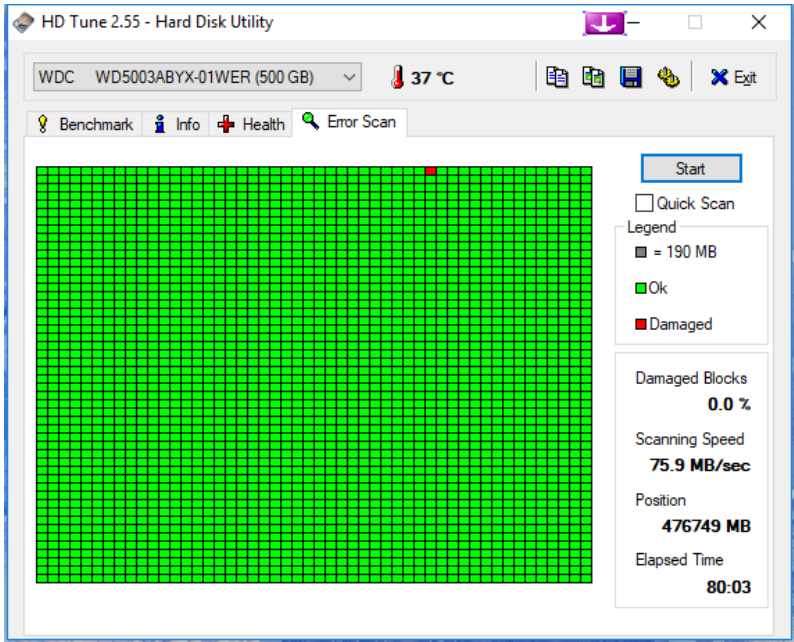


Figure 4.8: A disk surface-scan interface. The grid summarizes tested regions and reported read errors; it is not a raw-byte sector view or a verification of forensic-image integrity.

Forensic work normally examines a verified image rather than opening evidence media. At the logical level, an HDD can contain partitions, volumes, allocated files, deleted content, unallocated space, and metadata. At the physical level, the

structure explains why sector numbers and offsets can locate data independently of a filename. Damage, firmware problems, wear, and improper handling can affect what can be recovered, so preservation decisions should protect the original device and document its condition before examination.

Although solid-state drives use different physical technology, they present many of the same logical evidence concepts. Their controller behavior, wear leveling, and TRIM operations can make deleted-data recovery less predictable. Section 4.5 next examines the logical organization of storage.

4.5 Partitions, volumes, and drive letters

A partition divides a physical storage device into logical regions. A volume is storage made available to an operating system. A drive letter, such as **C:** or **D:** in Windows, is a convenient operating-system name for a mounted volume; it is not itself a physical disk. These distinctions help an examiner avoid confusing an entire source device with only one of its logical regions.

Figure 4.9 provides a conceptual view: one disk can contain multiple partitions, and each partition can provide a volume to the operating system. The illustration simplifies modern storage configurations, but it establishes the basic relationship needed to interpret later forensic views.

Figure 4.10 shows how Windows presents volumes to a user. The displayed names, drive letters, and free-space values are useful leads, but they do not establish the physical-device identity, partition boundaries, or forensic integrity of the data.

In Windows, right-click the Start button (or press **Win+X**) and choose **Disk Management**, as shown in Figure 4.11. Do not select **Device Manager**: it is useful for hardware-device information but does not provide the partition-and-volume view used in this section.

Figure 4.12 correlates physical disks with their partitions and volumes in Windows Disk Management. Read the disk number, capacity, partition size, status, and drive letter together. Forensic work should normally use an acquired image and a dedicated analysis tool; a live disk-management display is only a non-destructive laboratory orientation aid.

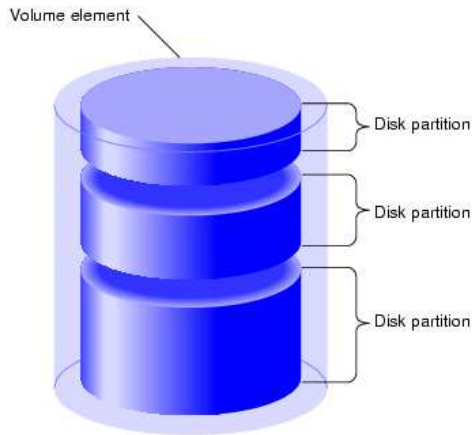


Figure 4.9: One physical disk divided into logical partitions. A partition can contain a volume that the operating system makes available to users.

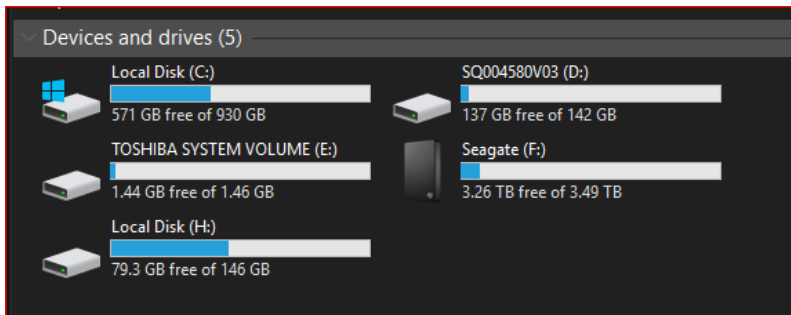


Figure 4.10: Windows Explorer displays mounted volumes with drive letters and available space. These labels are logical operating-system views, not physical-device identifiers.

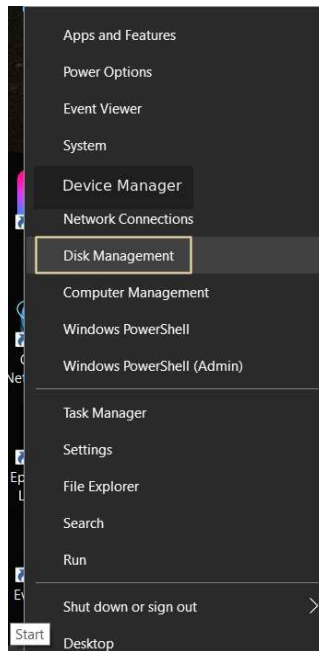


Figure 4.11: Opening Windows Disk Management from the Start-button context menu. Select **Disk Management**, not **Device Manager**, to view disks, partitions, and volumes.

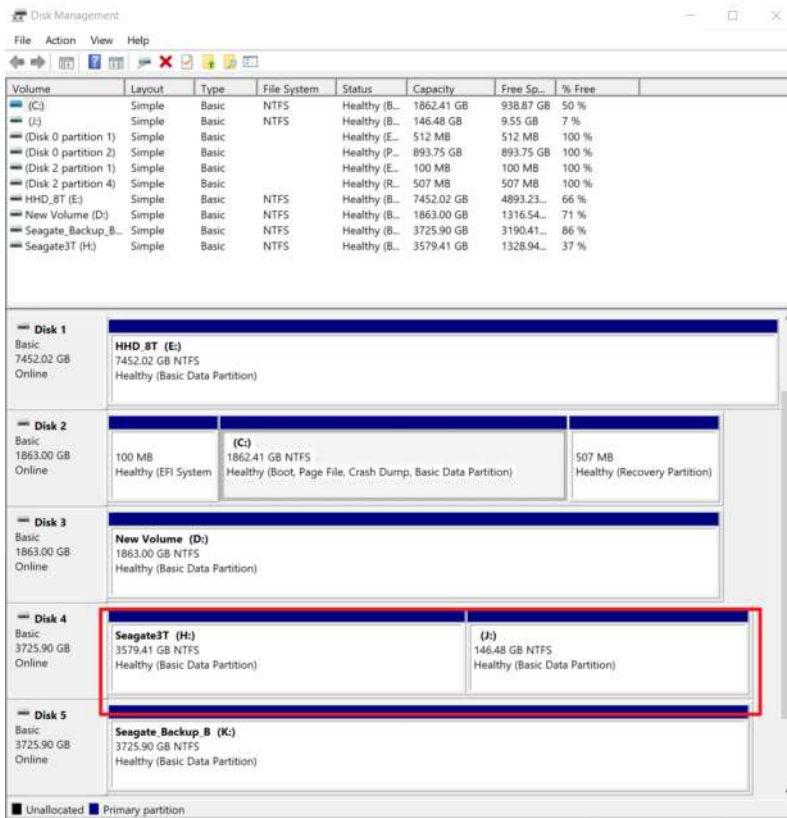


Figure 4.12: Windows Disk Management correlates physical disks, partitions, volumes, sizes, and drive letters. Use it only for authorized, non-destructive laboratory observation.

4.6 Booting process and partition-table overview

A partition table records how a device’s partitions are organized. The two common formats are the Master Boot Record (MBR) and GUID Partition Table (GPT). Table 4.2 provides a high-level comparison before the two structures are examined in detail. The firmware pairing is typical rather than absolute: systems can support compatibility modes, so an examiner should identify the actual firmware mode and partition structures from the evidence.

Table 4.2: High-level comparison of MBR and GPT partition tables [UEFI Forum, 2024, Microsoft, 2026a].

Partition table	Typical firmware pairing	Key characteristics
MBR	Legacy BIOS	Up to four primary partitions; practical 2 TiB limit with 512-byte logical sectors.
GPT	Modern UEFI	Primary and backup metadata; supports large disks; commonly 128 partition entries in Windows.

The boot process provides important context for an examination because it connects firmware settings, a selected storage device, boot code, and the running operating system. The exact sequence depends on whether a device uses the older BIOS/MBR approach or the modern UEFI approach, and on its operating-system configuration.

Legacy BIOS and MBR booting. After power is applied, firmware performs initial hardware checks, commonly called the power-on self-test (POST). In a traditional PC, the Basic Input/Output System (BIOS) initializes essential devices and follows its configured boot order. BIOS firmware was historically stored in ROM or flash memory, while its configuration was commonly stored in CMOS-backed memory. The firmware selects a boot device and transfers control to the boot code in that device’s MBR. The MBR code normally identifies an active partition and transfers control to its volume boot record, which begins the operating system’s boot-manager and loader sequence [Microsoft, 2026b].

Figure 4.13 illustrates this legacy hardware-level path from the power supply

through firmware and the CPU to a selected storage device. It is a conceptual BIOS/MBR model; modern UEFI systems do not normally bootstrap an operating system by executing MBR code.

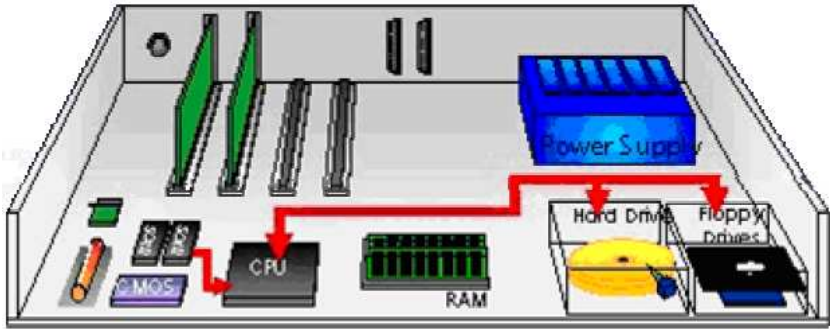


Figure 4.13: Conceptual legacy BIOS boot path. After power-on and firmware initialization, BIOS selects a boot device and can transfer control to code in its MBR. Modern UEFI systems follow a different path.

System and boot volumes. In Windows terminology, the *system* volume contains the files needed to start Windows, such as boot-manager files and boot configuration data. The *boot* volume contains the installed Windows operating-system files, including the kernel. Despite the names, the system volume starts the loading process and the boot volume contains the operating system being loaded; they can be the same partition or separate partitions [Microsoft, 2026b].

Modern UEFI booting. UEFI firmware initializes hardware, consults boot entries stored in nonvolatile firmware memory, and typically loads an EFI boot-manager program from the EFI System Partition (ESP). GPT is common with UEFI, but the partitioning scheme and firmware mode must be established from the evidence rather than assumed. The boot manager selects an operating system and loader; the loader initializes the kernel, which then starts drivers, services, and user-mode processes [UEFI Forum, 2024, Microsoft, 2026b].

Forensic questions may therefore include which firmware mode and boot order were configured, which disk and partition were selected, whether the MBR, volume boot record, EFI System Partition, boot configuration, or boot files changed, and whether an attached device or recovery environment could have affected the system state. Record such observations without changing firmware settings or booting an evidence device.

The boot-options view in Figure 4.14 emphasizes that firmware configuration can affect which device starts the system. Record such observations carefully and do not change boot settings merely to inspect them.

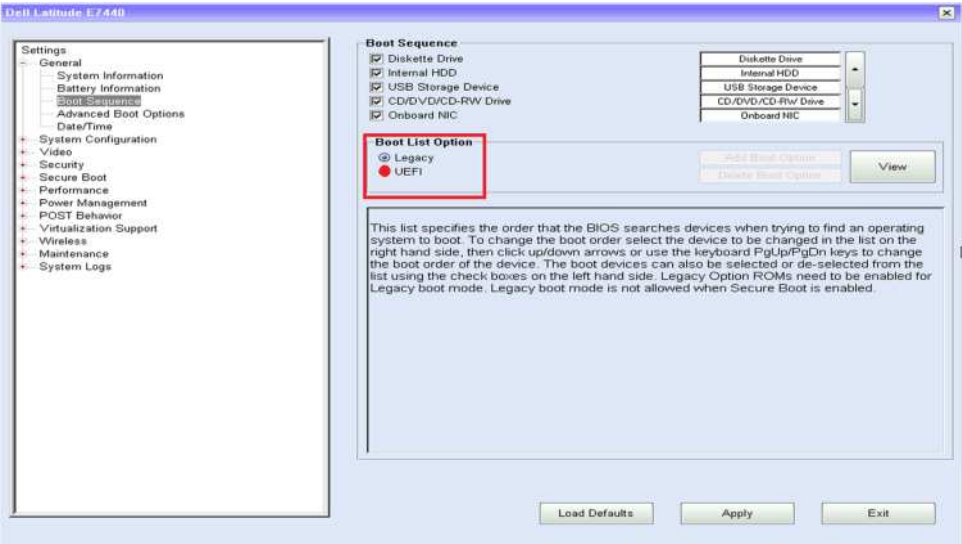


Figure 4.14: A firmware boot-options view. Boot configuration can provide useful context but should be inspected without changing the system state.

4.7 Master Boot Record

The Master Boot Record occupies the first logical sector of a legacy BIOS boot disk, traditionally a 512-byte sector. It contains bootstrap code, a partition table with four 16-byte entries, and the traditional 0x55AA boot signature. Figure 4.15 shows the conventional byte layout, including the disk signature and reserved bytes. An extended partition can link to additional logical partitions. During an examination, report the offset, raw bytes, and interpretation used to identify these structures [NeoSmart Technologies, 2026].

Illustrative MBR views. The following screenshots demonstrate how a hexadecimal viewer can expose MBR structures. They are included to help readers recognize the interfaces and the reasoning steps, not to prescribe a live-evidence

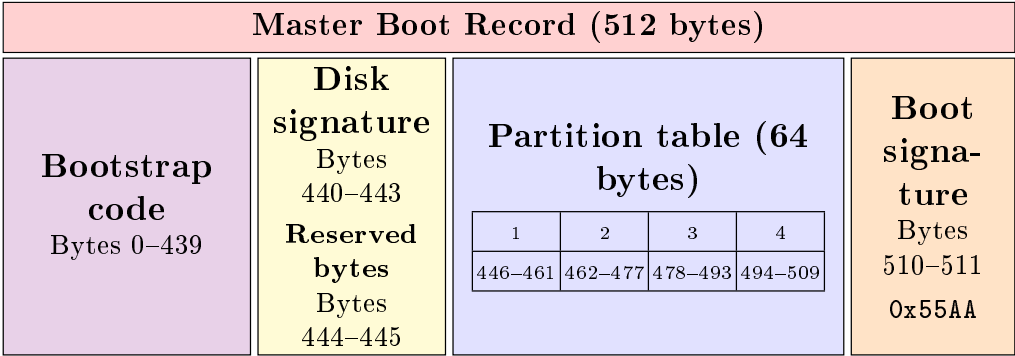


Figure 4.15: Conventional layout of a 512-byte Master Boot Record. The byte ranges identify the structures an examiner can locate in the first logical sector of a legacy BIOS boot disk.

workflow. Work on a verified copy whenever possible.

Tool choices. Both HxD and WinHex can view and edit hexadecimal data, including raw disk data. The screenshots in this section use WinHex, but the interpretation principles apply to either tool and to comparable hexadecimal viewers. Editing changes evidence: inspect a verified forensic copy or use a demonstrably read-only workflow, and record the tool, version, source, and mode used.

Figure 4.16 shows how to locate WinHex, and the red rectangle in Figure 4.17 identifies the **Open Disk** command used to begin opening a physical device.

After choosing **Open Disk**, WinHex displays the **Select Disk** dialog shown in Figure 4.18. The red rectangle highlights the physical storage device, not a logical volume or partition. In this laboratory example, HD0 is a VMware virtual disk presented to the virtual machine as a storage device. Before proceeding, compare the displayed size and identifier with the acquisition record.

Before examining the disk bytes, configure the hexadecimal display so that offsets and byte groups are easy to read. Figure 4.19 shows the **Options** → **General** path. In the General Options dialog, shown in Figure 4.20, enable hexadecimal offsets and use 16 bytes per line. These are display settings: they affect how bytes are presented, not the bytes stored on the disk.

Figure 4.21 shows how to display data-interpreter controls. Logical Block Ad-

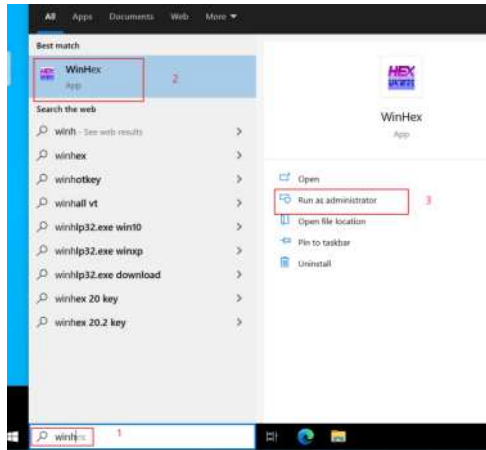


Figure 4.16: Launching WinHex.

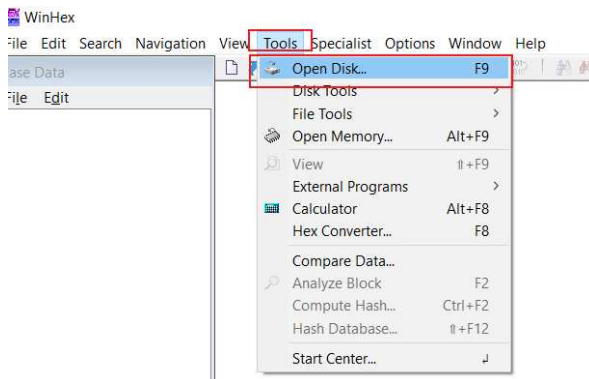


Figure 4.17: The **Open Disk** command in WinHex, highlighted in red. This is the menu step used to begin opening a physical device.